Natural Language Processing and LLMs Lecture 04: Language Models; Recurrent Neural Networks; Sequence Tagging

Valentin Malykh, MWS AI

M W S AI

Autumn 2026

The course is delivered at MIPT, HSE (Moscow), ITMO (Saint Petersburg), and IITU (Almaty)

Content

1. Language Models (LMs)
2. N-gram Language Models
3. Language Models Based on Recurrent Neural Networks
4. Sequence Tagging

Content

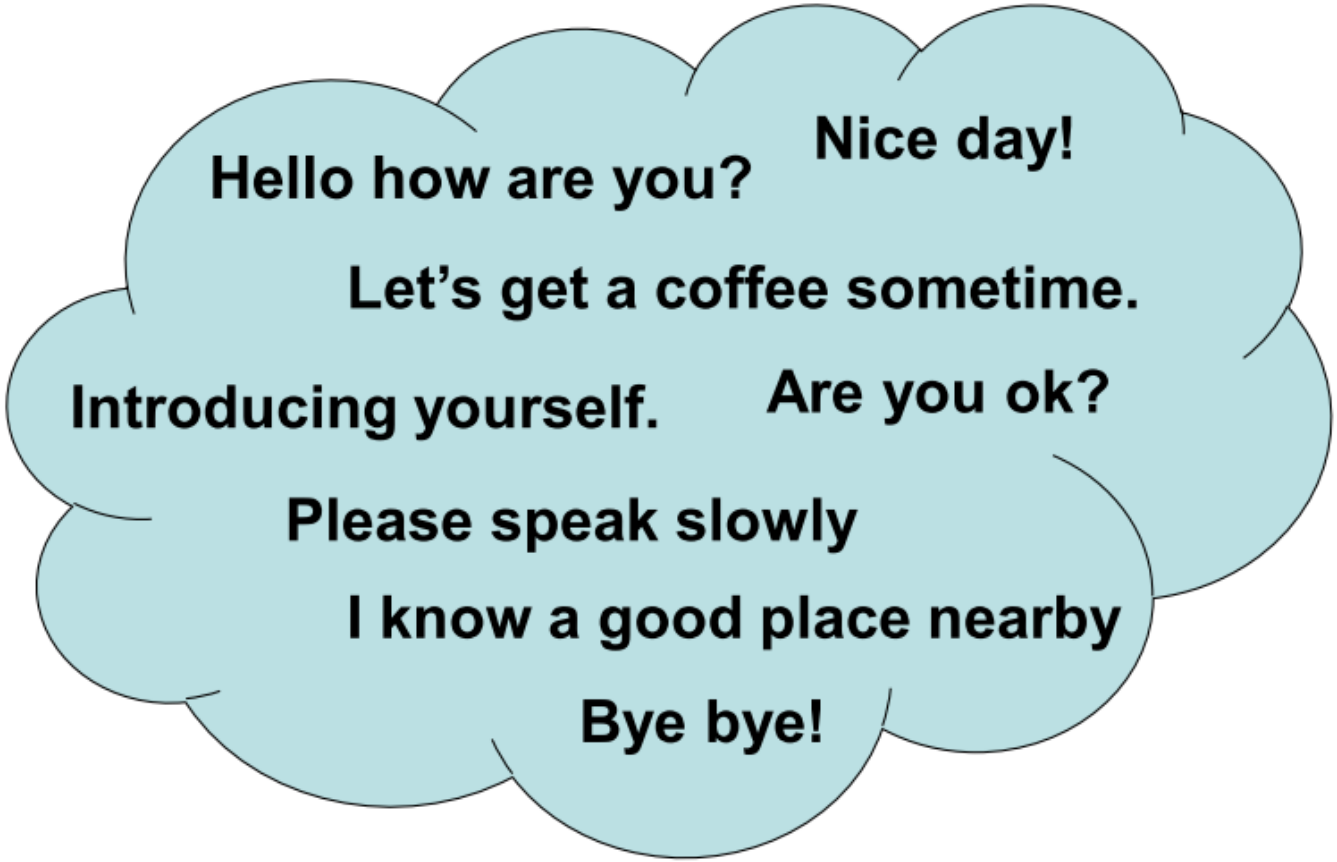
1 Language Models (LMs)

2 N-gram Language Models

3 Language Models Based on Recurrent Neural Networks

4 Sequence Tagging

How can we define a language? (Recap)



How can we define a language? (Recap)

— The set theory approach

- A language can be defined as the set of sentences which can be accepted by the speakers of that language.
- It is not possible to define a natural language by enumerating all the sentences, because the number of sentences in a natural language is infinite.
- Two feasible ways to define a language with infinite sentences:
  - By a Grammar
  - By an Automaton

How can we define a language?

— The probabilistic approach

- A language can also be defined as a probabilistic distribution over all the possible sentences.
- A statistical language model is a probability distribution over sequences of words (sentences) in a given language L:

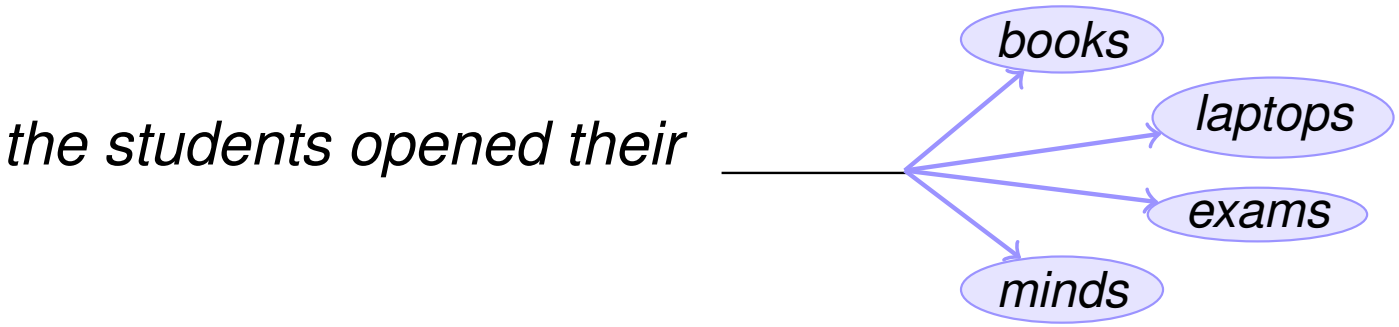
$$\sum_{s \in V^+} P_{LM}(s) = 1$$

- Or:

$$\sum_{\substack{s = w_1 w_2 \ldots w_n \\ w_i \in V, n > 0}} P_{LM}(s) = 1$$

Language Modeling

- Language Modeling is the task of predicting what word comes next.



- More formally: given a sequence of words $\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)}$, compute the probability distribution of the next word $\boldsymbol{x}^{(t+1)}$:

$$P\left(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}\right)$$

- where $\boldsymbol{x}^{(t+1)}$ can be any word in the vocabulary $V = \{\boldsymbol{w}_1, \ldots, \boldsymbol{w}_{|V|}\}$
- A system that does this is called a Language Model.

Language Modeling

- You can also think of a Language Model as a system that assigns probability to a piece of text.
- For example, if we have some text $x^{(1)}, \ldots, x^{(T)}$, then the probability of this text (according to the Language Model) is:

$$P\left(x^{(1)}, \ldots, x^{(T)}\right) = P\left(x^{(1)}\right) \times P\left(x^{(2)} \mid x^{(1)}\right) \times \ldots \times P\left(x^{(T)} \mid x^{(T-1)}, \ldots, x^{(1)}\right)$$

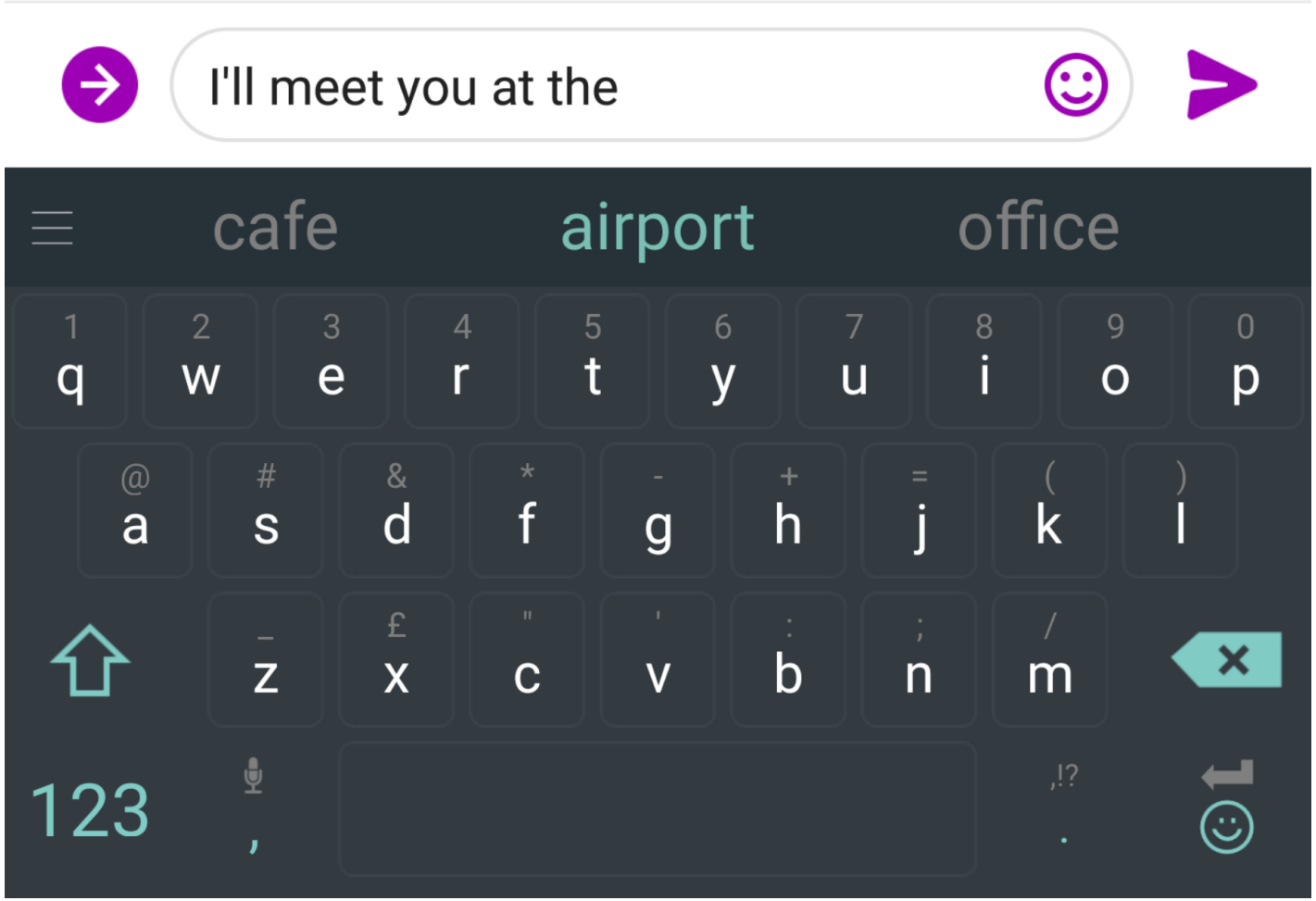
$$= \prod_{t=1}^{T} P\left(x^{(t)} \mid x^{(t-1)}, \ldots, x^{(1)}\right)$$

- This is what our LM provides

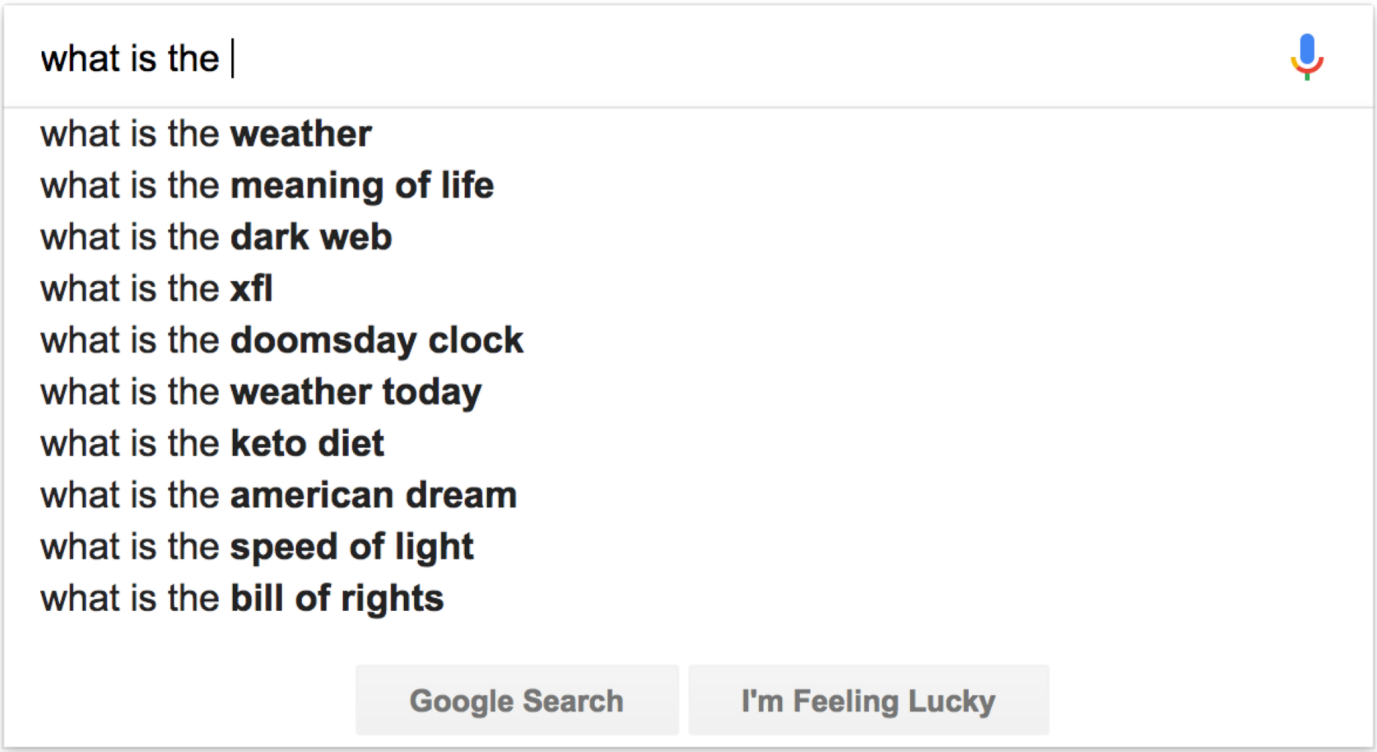
LMs are extremely powerful tools

- The definition of LMs is simple but LMs are extremely powerful tools in NLP
- A number of NLP problems could be viewed as variations of language modelling problems, and can be solved by LMs.
  - Text generation / Data to text / Image captioning / Text summarization
  - Machine translation
  - Speech Recognition
  - Reading Comprehension
  - POS tagging / Entity recognition / Parsing
- Any task which could be transformed into a sequential problem is suitable for LMs

You use Language Models every day!



You use Language Models every day!



NLP tasks as Conditional LMs

x "input"	w "text output"
An author	A document written by that author
A topic label	An article about that topic
{SPAM, NOT_SPAM}	An email
A sentence in French	Its English translation
A sentence in English	Its French translation
A sentence in English	Its Chinese translation
An image	A text description of the image
A document	Its summary
A document	Its translation
Meteorological measurements	A weather report
Acoustic signal	Transcription of speech
Conversational history + database	Dialogue system response
A question + a document	Its answer
A question + an image	Its answer

Content

1 Language Models (LMs)

2 N-gram Language Models

3 Language Models Based on Recurrent Neural Networks

4 Sequence Tagging

n-gram Language Models

- Question: How to learn a Language Model?
- Answer (pre-Deep Learning): learn an n-gram Language Model!
- Definition: A n-gram is a chunk of *n* consecutive words.
  - unigrams: “the”, “students”, “opened”, “their”
  - bigrams: “the students”, “students opened”, “opened their”
  - trigrams: “the students opened”, “students opened their”
  - 4-grams: “the students opened their”
- Idea: Collect statistics about how frequent different n-grams are, and use these to predict next word.

n-gram Language Models

- First we make a Markov assumption: $\boldsymbol{x}^{(t+1)}$ depends only on the preceding *n-1* words.

$$P\left(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}\right) = P\left(\boldsymbol{x}^{(t+1)} \mid \overbrace{\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}}^{n\text{-1 words}}\right) \quad \text{(assumption)}$$

$$= \frac{P\left(\boldsymbol{x}^{(t+1)}, \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}\right)}{P\left(\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}\right)} \quad \text{(definition of conditional prob)}$$

prob of a *n*-gram → $P\left(\boldsymbol{x}^{(t+1)}, \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}\right)$

prob of a (*n*-1)-gram → $P\left(\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}\right)$

- **Question:** How do we get these *n*-gram and (*n*-1)-gram probabilities?
- **Answer:** By counting them in some large corpus of text!

$$\approx \frac{\text{count}\left(\boldsymbol{x}^{(t+1)}, \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}\right)}{\text{count}\left(\boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(t-n+2)}\right)} \quad \text{(statistical approximation)}$$

Bigram LM: Parameters

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Bigram LM: Probabilities

- Normalize by unigrams:

$$P(w_n \mid w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

	i	want	to	eat	chinese	food	lunch	spend
$C(w)$	2533	927	2417	746	158	1093	341	278
i	.002	.33	0	.0036	0	0	0	.00079
want	.0022	0	.66	.0011	.0065	.0065	.0054	.0011
to	.00083	0	.0017	.28	.00083	0	.0025	.087
eat	0	0	.0027	0	.021	.0027	.056	0
chinese	.0063	0	0	0	0	.52	.0063	0
food	.014	0	.014	0	.00092	.0037	0	0
lunch	.0059	0	0	0	0	.0029	0	0
spend	.0036	0	.0036	0	0	0	0	0

n-gram Language Models: Example

Suppose we are learning a 4-gram Language Model.

~~*as the proctor started the clock,*~~ *the students opened their* _____

~~*as the proctor started the clock,*~~: discard

the students opened their: condition on this

$$P(\boldsymbol{w} \mid \text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

- "students opened their" occurred 1000 times
- "students opened their books" occurred 400 times
  - $\rightarrow P(\text{books} \mid \text{students opened their}) = 0.4$
- "students opened their exams" occurred 100 times
  - $\rightarrow P(\text{exams} \mid \text{students opened their}) = 0.1$

Should we have discarded the "proctor" context?

Sparsity Problems with n-gram Language Models

Sparsity Problem 1

Problem: What if *"students opened their **w**"* never occurred in data? Then **w** has probability 0!

(Partial) Solution: Add small δ to the count for every $\boldsymbol{w} \in V$. This is called *smoothing*.

$$P(\boldsymbol{w} \mid \text{students opened their}) = \frac{\text{count(students opened their } \boldsymbol{w})}{\text{count(students opened their)}}$$

Sparsity Problem 2

Problem: What if *"students opened their"* never occurred in data? Then we can't calculate probability for *any* **w**!

(Partial) Solution: Just condition on *"opened their"* instead. This is called *backoff*.

Note: Increasing n makes sparsity problems *worse*. Typically we can't have n bigger than 5.

Storage Problems with n-gram Language Models

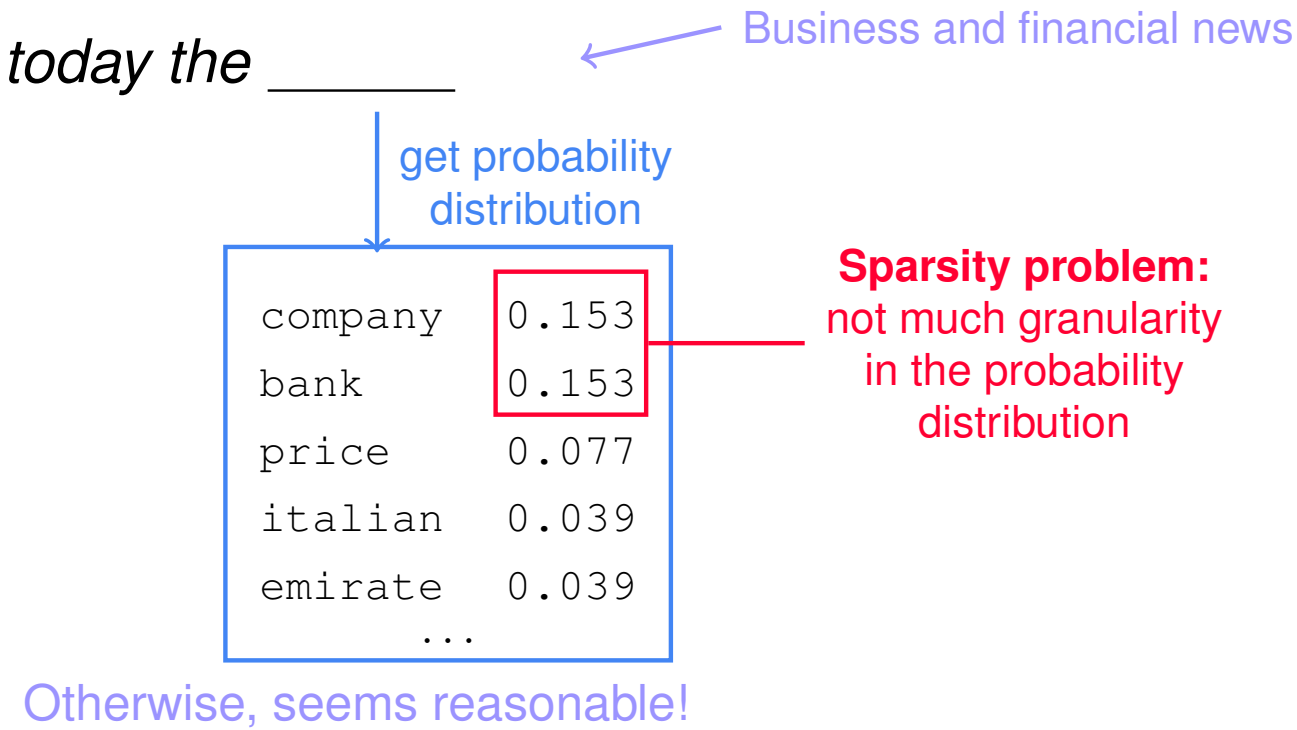
Storage: Need to store count for all *n*-grams you saw in the corpus.

$$P(\boldsymbol{w} \mid \text{students opened their}) = \frac{\text{count(students opened their } \boldsymbol{w})}{\text{count(students opened their)}}$$

Increasing n or increasing corpus increases model size!

n-gram Language Models in practice

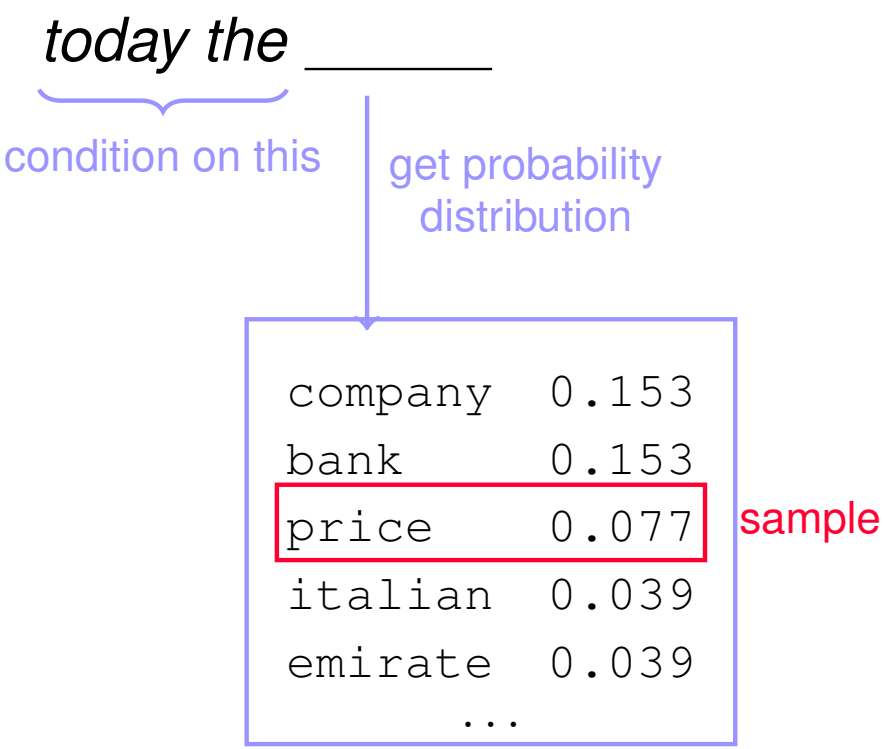
- You can build a simple trigram Language Model over a 1.7 million word corpus (Reuters) in a few seconds on your laptop*



* Try for yourself: nlpforhackers.io/language-models

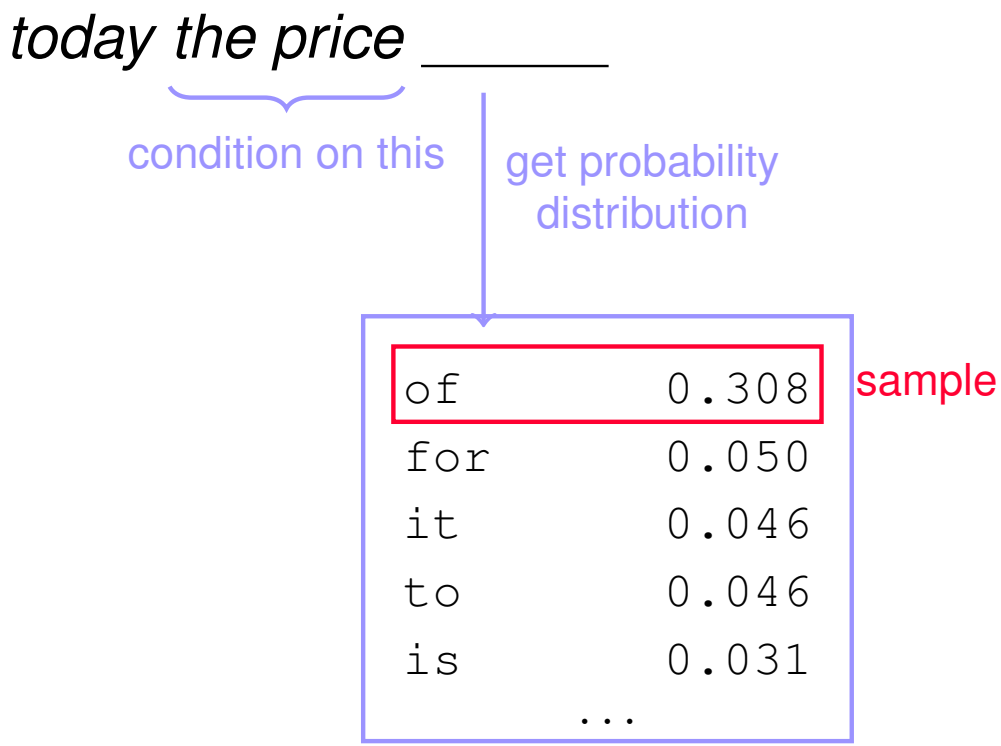
Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.



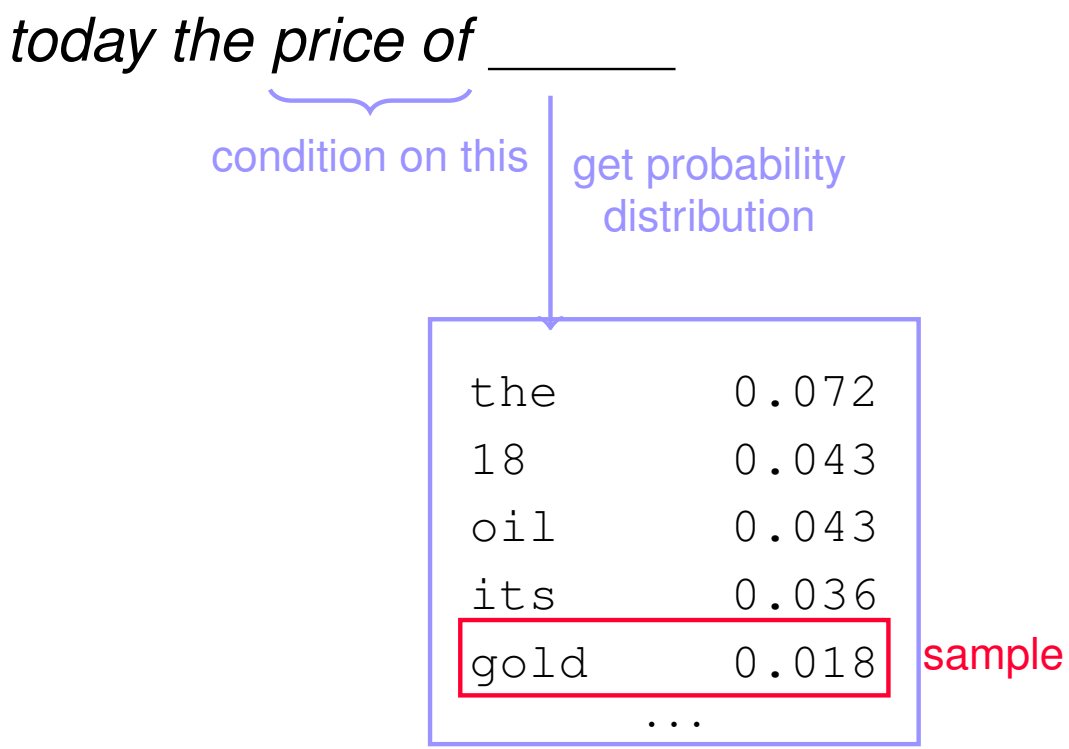
Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.



Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.



Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.

today the price of gold _____

Generating text with a n-gram Language Model

- You can also use a Language Model to generate text.

today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share .

- Surprisingly grammatical! . . . but incoherent. We need to consider more than three words at a time if we want to model language well.
- But increasing n worsens sparsity problem, and increases model size. . .

Content

1 Language Models (LMs)

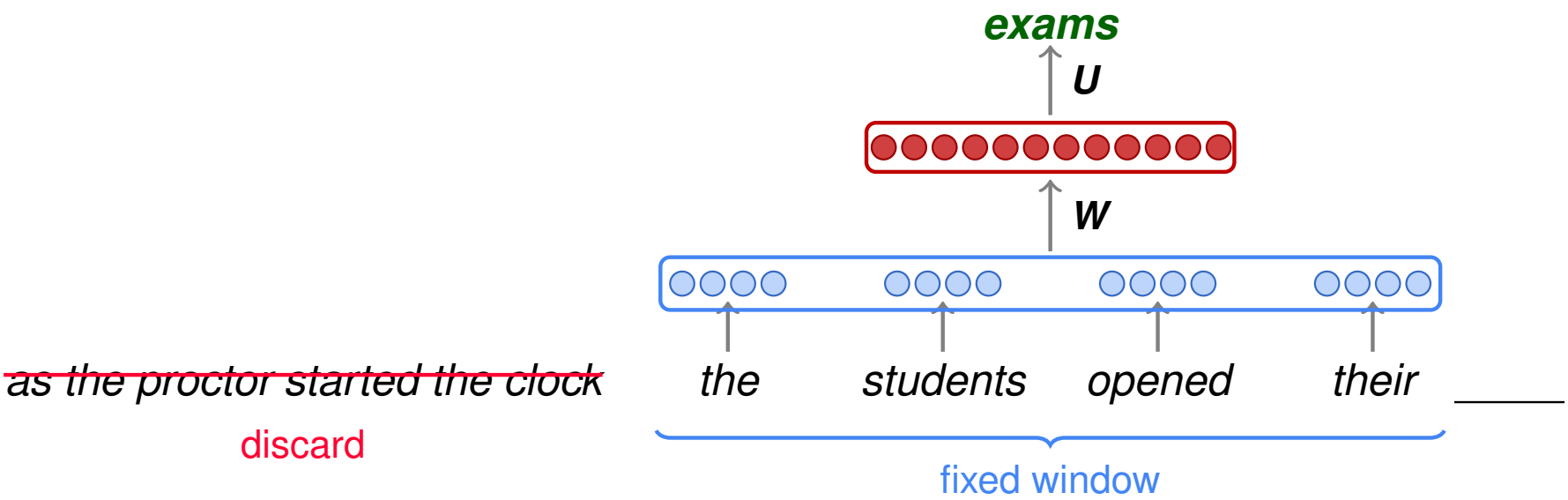
2 N-gram Language Models

3 Language Models Based on Recurrent Neural Networks

4 Sequence Tagging

How to build a *neural* Language Model?

- Recall the Language Modeling task:
  - Input: sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
  - Output: prob dist of the next word $P\left(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}\right)$
- How about a window-based neural model? Predict the next word from a fixed window of the last words:



A fixed-window neural Language Model

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

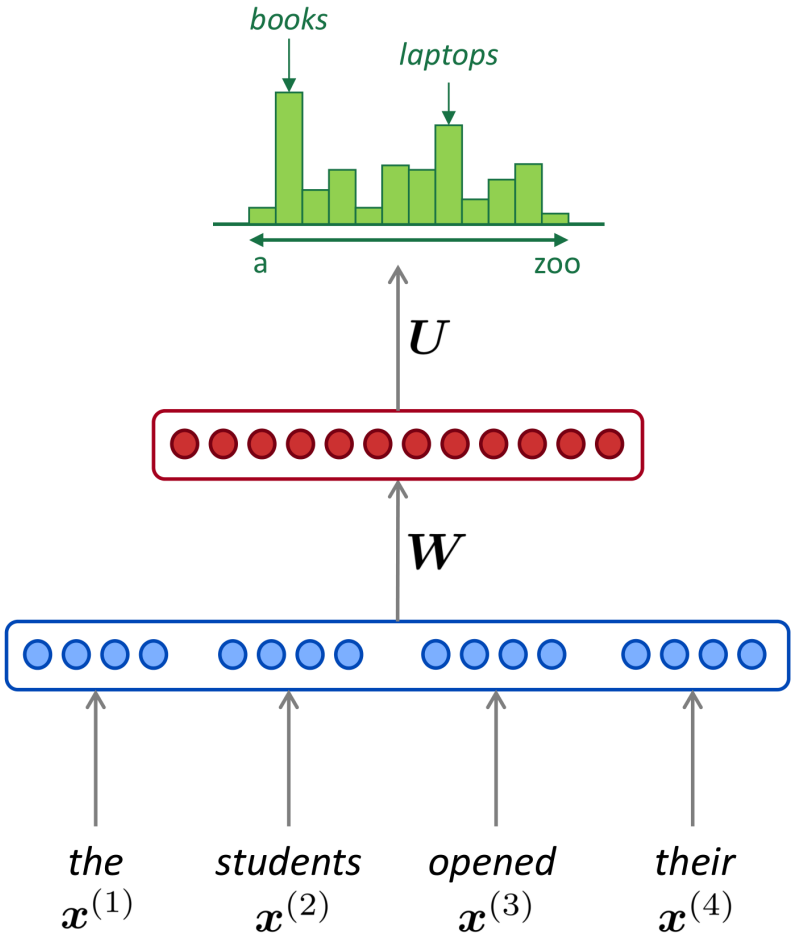
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

A fixed-window neural Language Model

Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

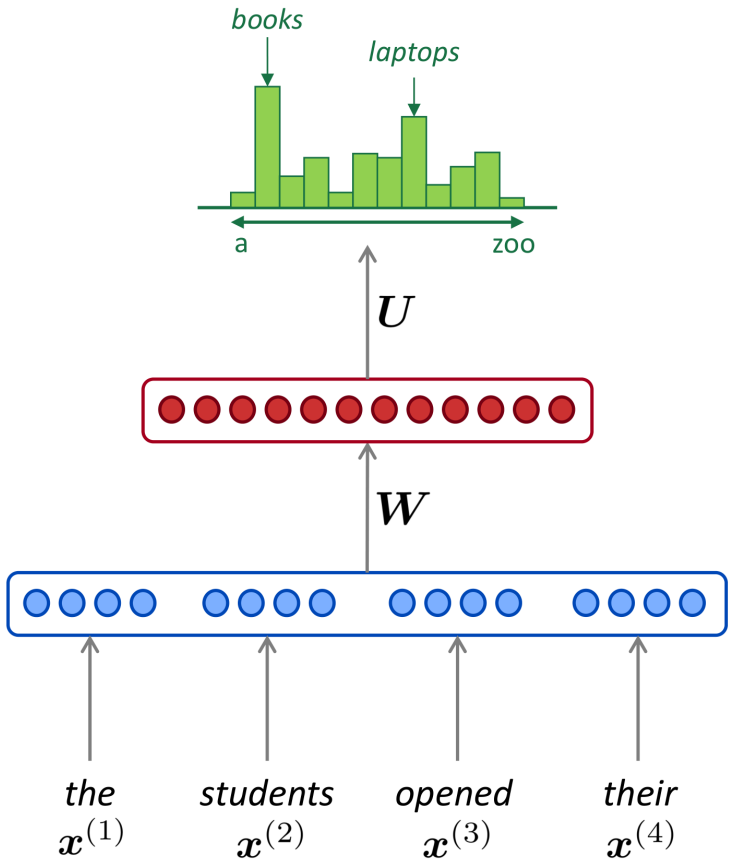
Improvements over *n*-gram LM:

- No sparsity problem
- Don't need to store all observed *n*-grams

Remaining **problems**:

- Fixed window is too small
- Enlarging window enlarges $\boldsymbol{W}$
- Window can never be large enough!
- $\boldsymbol{x}^{(1)}$ and $\boldsymbol{x}^{(2)}$ are multiplied by completely different weights in $\boldsymbol{W}$. No symmetry in how the inputs are processed.

We need a neural architecture that can process *any length input*



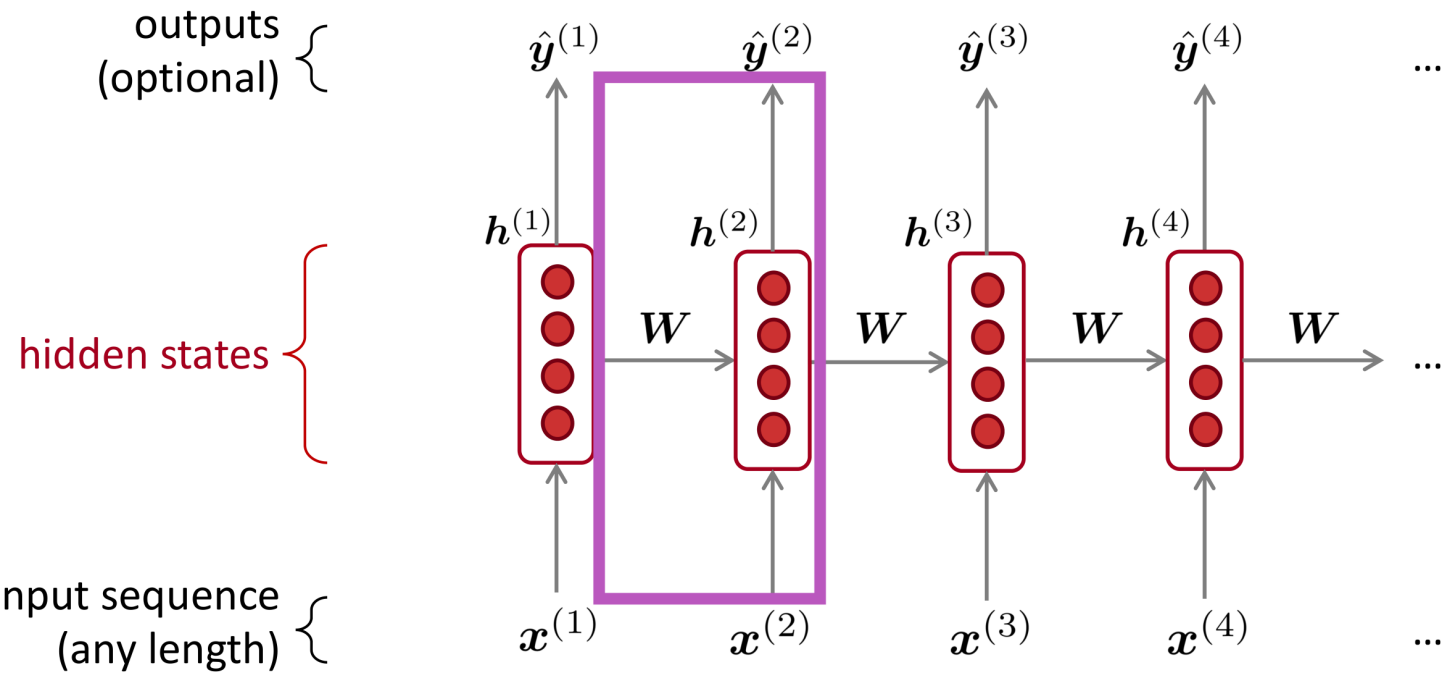
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Recurrent Neural Networks (RNN)

A family of neural architectures

Core idea:

Apply the same weights ***W*** *repeatedly*



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

A Simple RNN Language Model

output distribution

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}\left(\boldsymbol{U}\boldsymbol{h}^{(t)} + \boldsymbol{b}_2\right) \in \mathbb{R}^{|V|}$$

hidden states

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_e\boldsymbol{e}^{(t)} + \boldsymbol{b}_1\right)$$

$\boldsymbol{h}^{(0)}$ is the initial hidden state

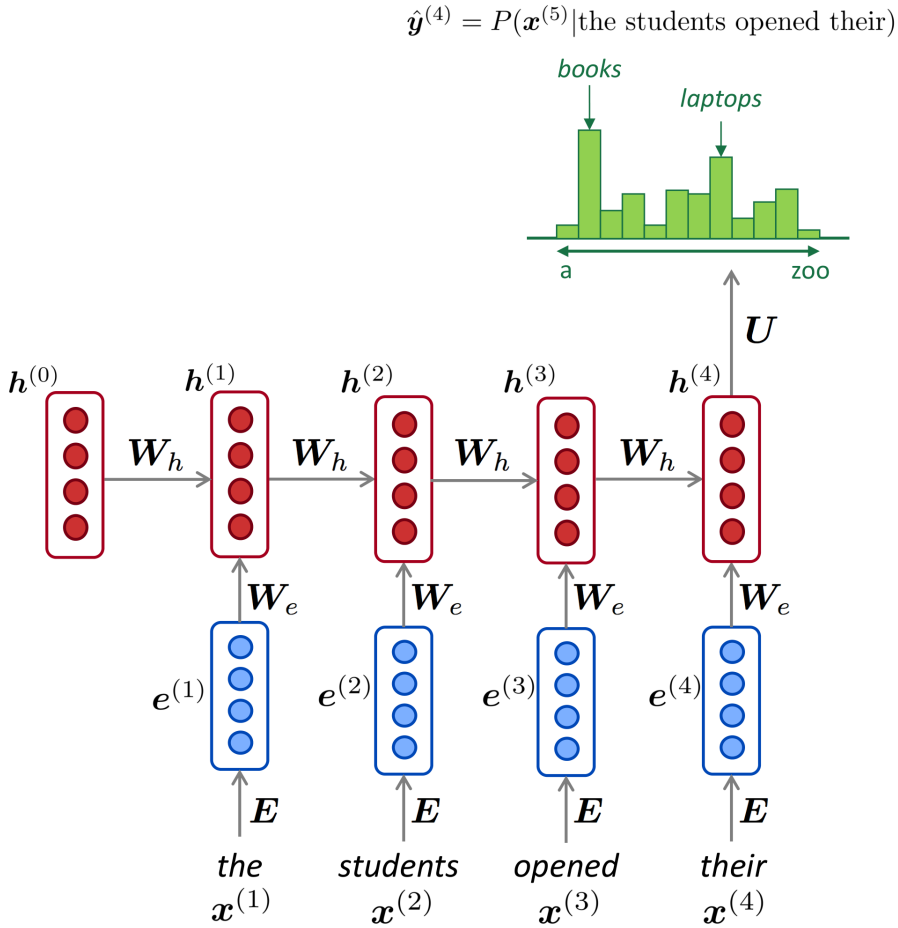
word embeddings

$$\boldsymbol{e}^{(t)} = \boldsymbol{E}\boldsymbol{x}^{(t)}$$

words / one-hot vectors

$$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$$

Note: this input sequence could be much longer, but this slide doesn't have space!



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

RNN Language Models

Advantages	Disadvantages
✓ Can process any length input	× Recurrent computation is slow
✓ Computation for step t can (in theory) use information from many steps back	× In practice, difficult to access information from many steps back
✓ Model size doesn't increase for longer input	
✓ Same weights applied on every timestep, so there is symmetry in how inputs are processed	

(More on these later in the course)

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Training an RNN Language Model

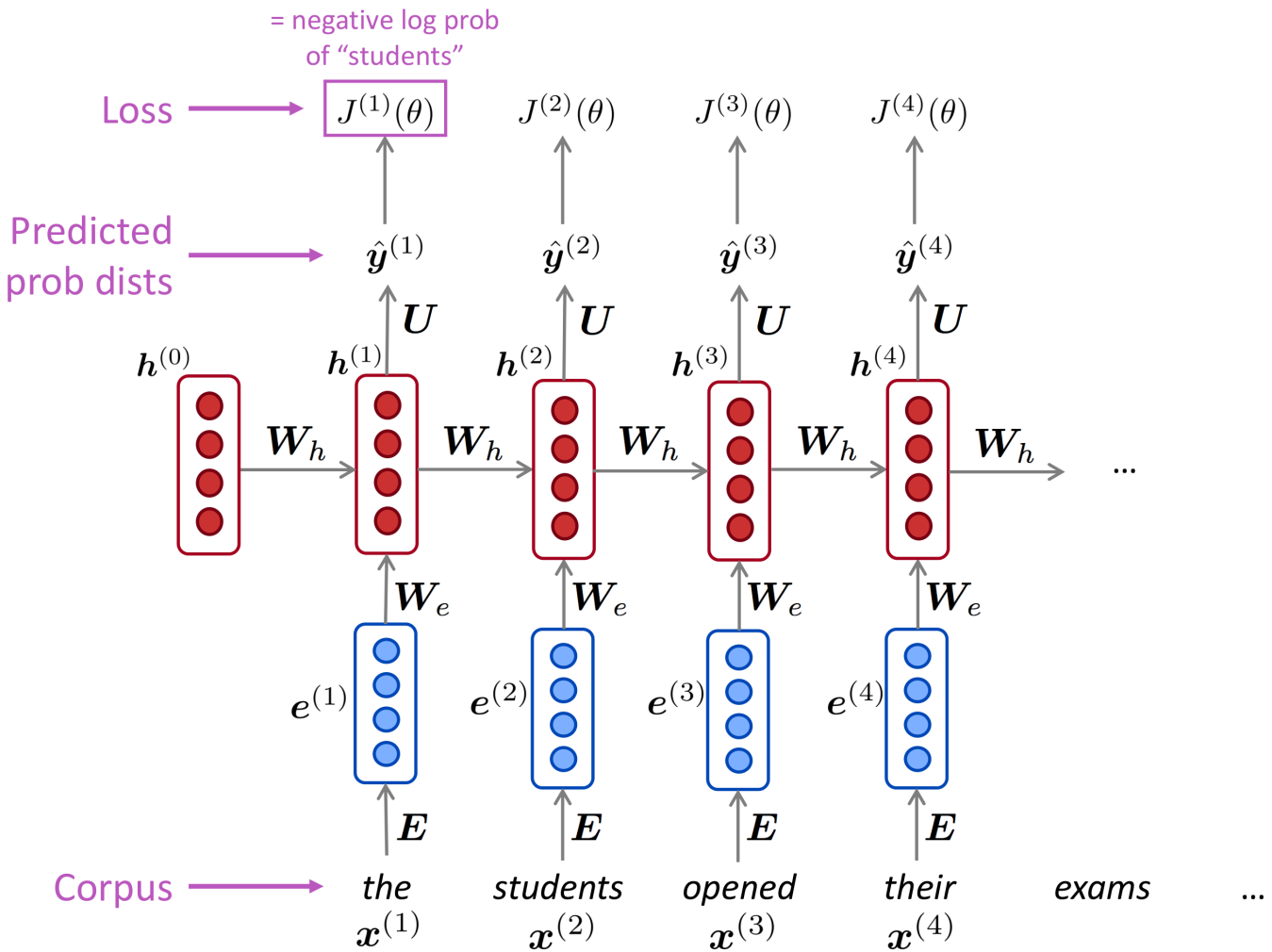
- Get a big corpus of text which is a sequence of words $x^{(1)}, \ldots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ for every step t
  - i.e. predict probability dist of every word, given words so far
- Loss function on step t is cross-entropy between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = -\sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = -\log \hat{y}_{x_{t+1}}^{(t)}$$

- Average this to get overall loss for entire training set:

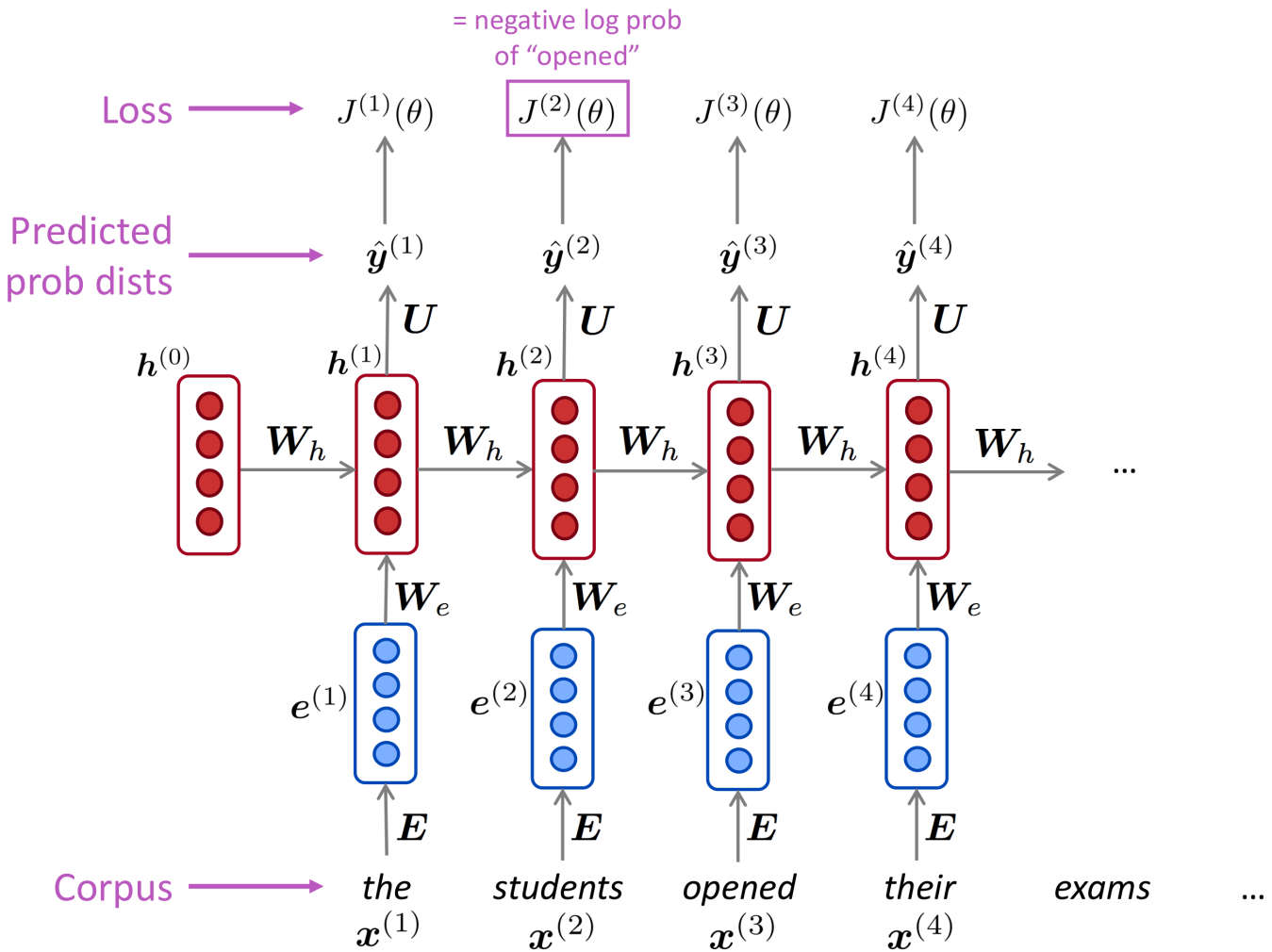
$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^{T} -\log \hat{y}_{x_{t+1}}^{(t)}$$

Training an RNN Language Model



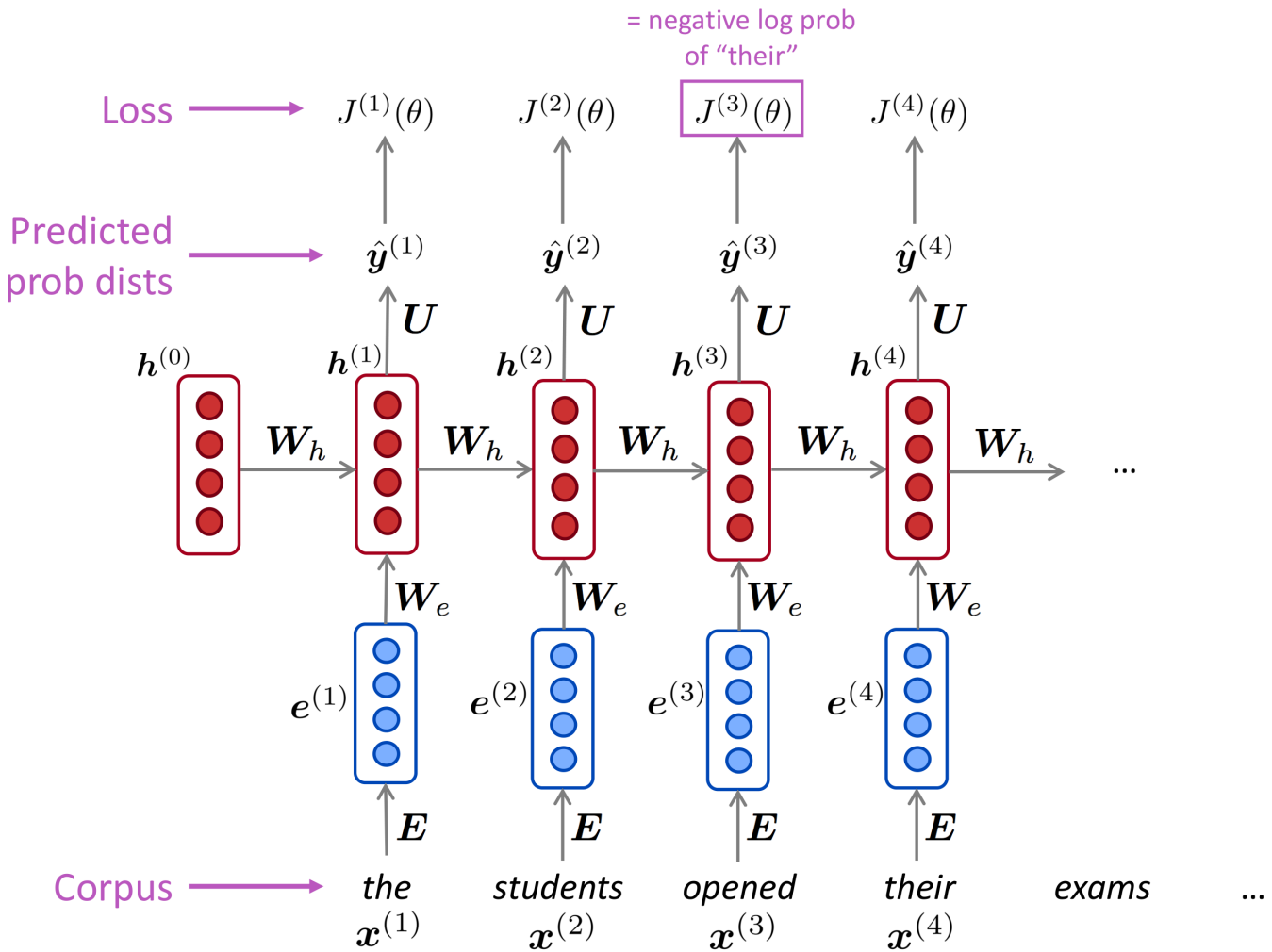
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Training an RNN Language Model



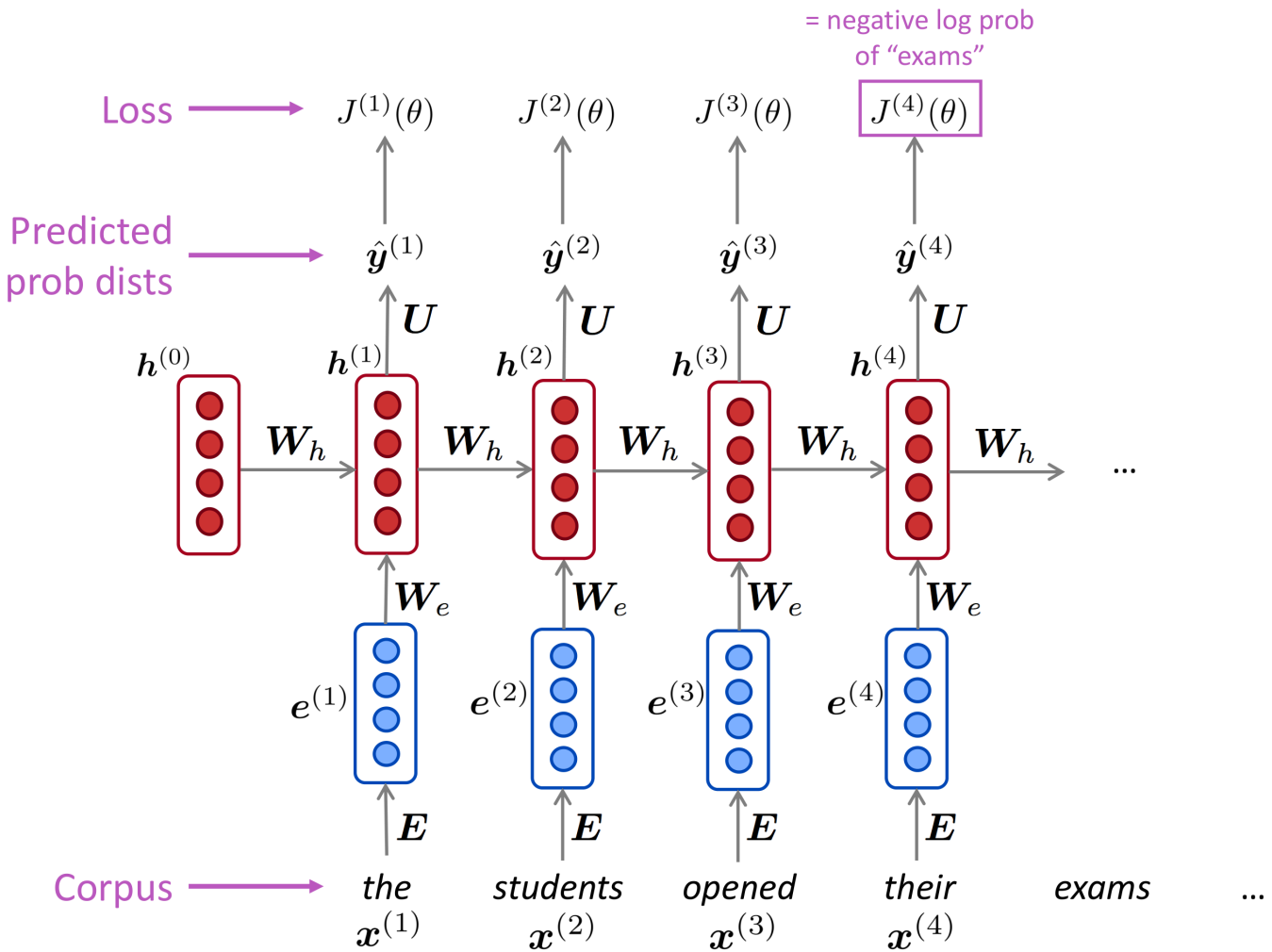
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Training an RNN Language Model



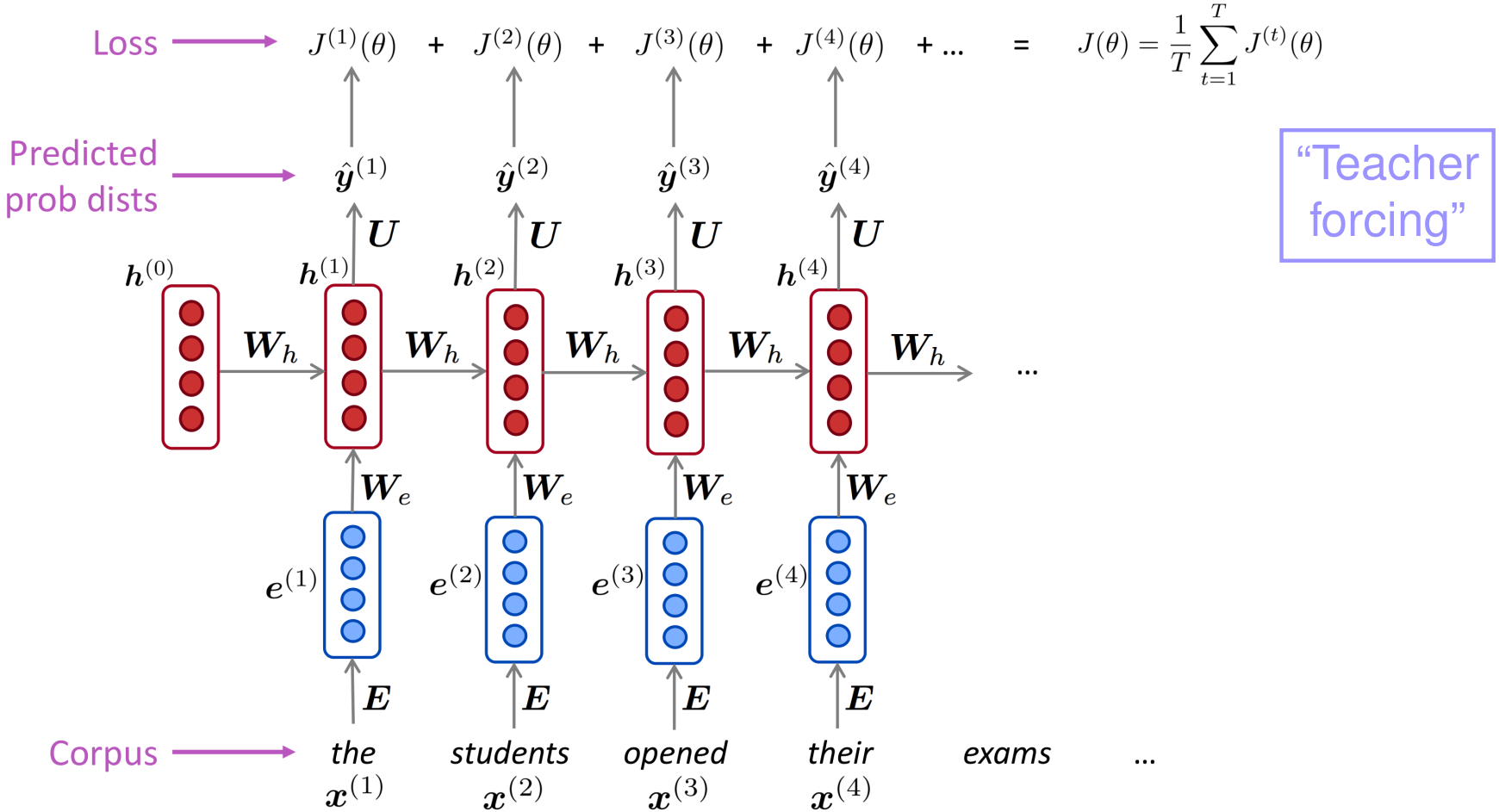
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Training an RNN Language Model



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Training an RNN Language Model

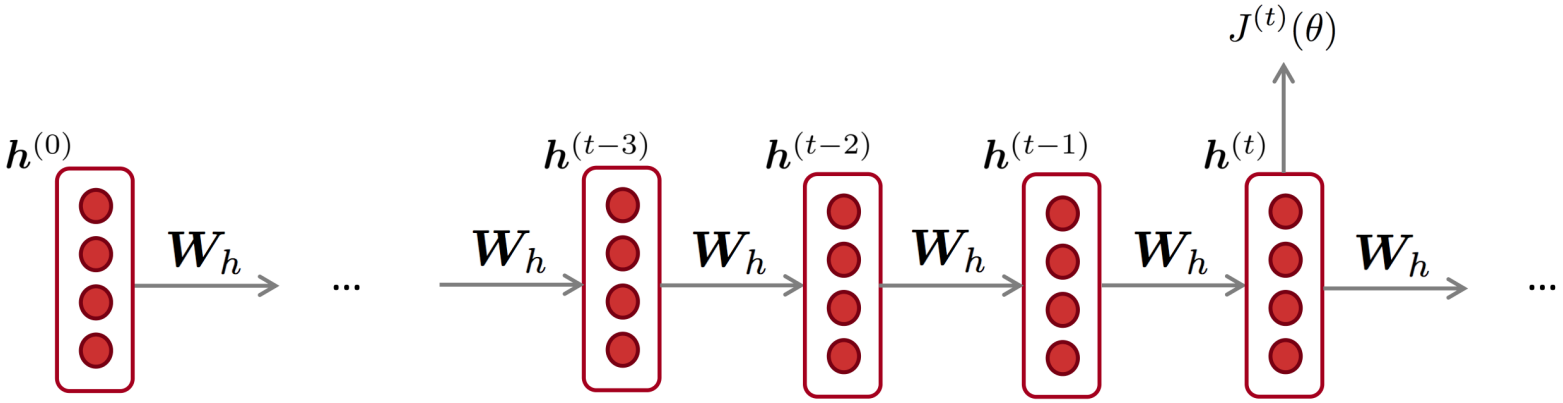


A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Training a RNN Language Model

- However: Computing loss and gradients across entire corpus is too expensive!
- In practice, consider $x^{(t)}$ as a sentence (or a document)
- Recall: Stochastic Gradient Descent allows us to compute loss and gradients for small chunk of data, and update
- Compute loss $J^{(t)}$ for a sentence (actually a batch of sentences), compute gradients and update weights. Repeat.

Backpropagation for RNNs



Question: What's the derivative of $J^{(t)}(\theta)$ w.r.t. the repeated weight matrix $\boldsymbol{W}_h$?

Answer:

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h} = \sum_{i=1}^{t} \left. \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h} \right|_{(i)}$$

"The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears"

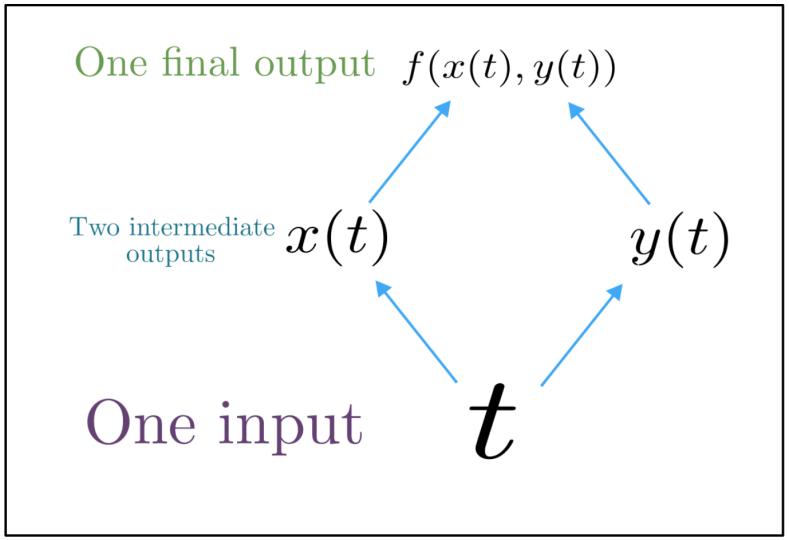
Why?

A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Multivariable Chain Rule

- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$



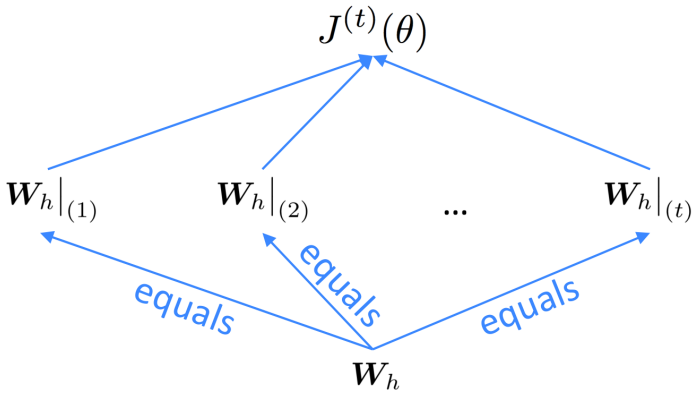
Source: Khan Academy: Multivariable chain rule, simple version

Backpropagation for RNNs: Proof sketch

- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f\big(x(t), y(t)\big)}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$

In our example:



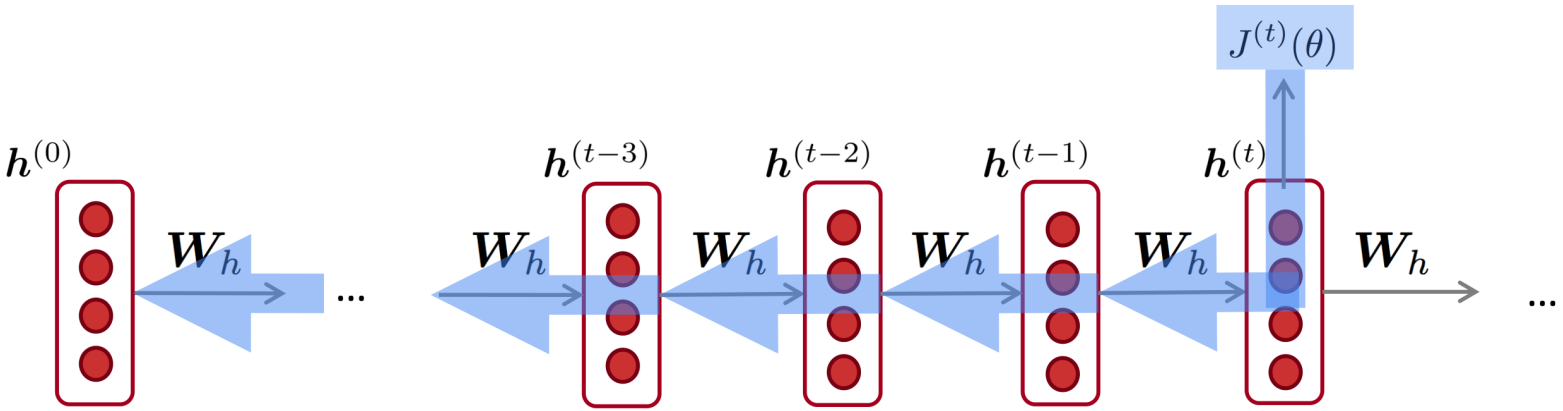
Apply the multivariable chain rule:

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h}\bigg|_{(i)} \boxed{\frac{\partial \boldsymbol{W}_h|_{(i)}}{\partial \boldsymbol{W}_h}} \quad (= 1)$$

$$= \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h}\bigg|_{(i)}$$

Source: Khan Academy: Multivariable chain rule, simple version

Backpropagation for RNNs



$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h} = \sum_{i=1}^{t} \left. \frac{\partial J^{(t)}}{\partial \boldsymbol{W}_h} \right|_{(i)}$$

Question: How do we calculate this?

Answer: Backpropagate over timesteps $i = t, \ldots, 0$, summing gradients as you go.
This algorithm is called
"backpropagation through time"
[Werbos, P.G., 1988, *Neural Networks* **1**, and others]

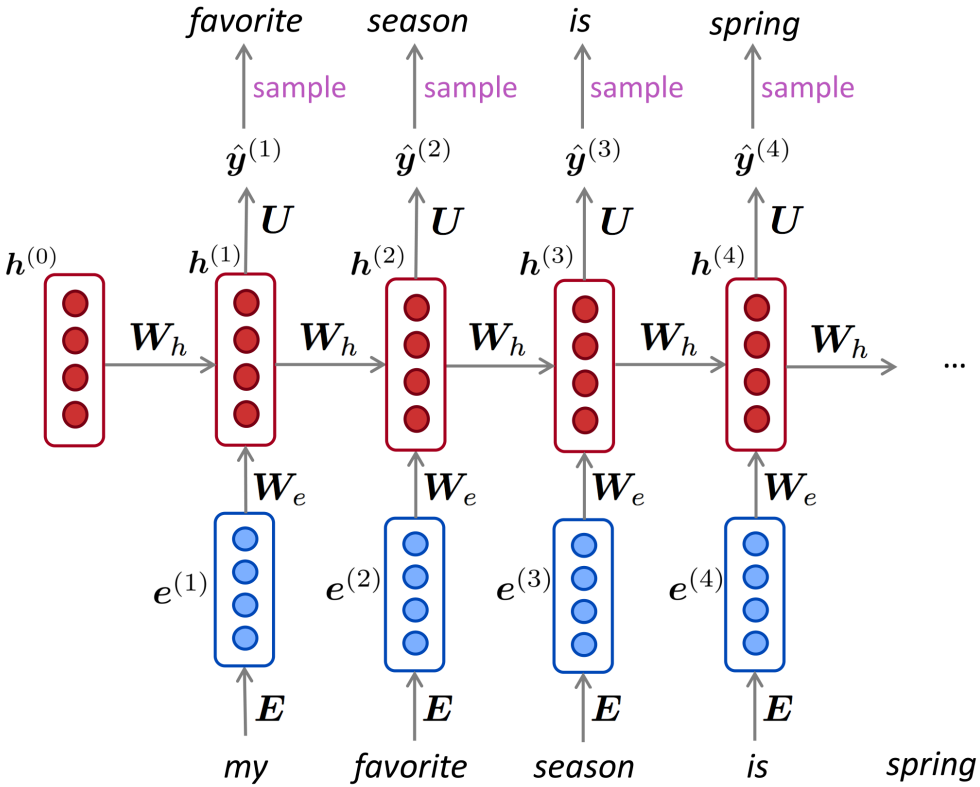
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Generating text with a RNN Language Model

Just like a n-gram Language Model, you can use a RNN Language Model to generate text by repeated sampling. Sampled output is next step's input.



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Generating text with a RNN Language Model

Let's have some fun!

- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on *Harry Potter*:



"Sorry," Harry shouted, panicking—"I'll leave those brooms in London, are they?"

"No idea," said Nearly Headless Nick, casting low close by Cedric, carrying the last bit of treacle Charms, from Harry's shoulder, and to answer him the common room perched upon it, four arms held a shining knob from when the spider hadn't felt it seemed. He reached the teams too.

Source: medium.com/deep-writing/harry-potter-written-by-artificial-intelligence

Generating text with a RNN Language Model

Let's have some fun!

- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on recipes:



```
     Title: CHOCOLATE RANCH BARBECUE
Categories: Game, Casseroles, Cookies, Cookies
     Yield: 6 Servings

     2 tb Parmesan cheese -- chopped
     1 c  Coconut milk
     3    Eggs, beaten

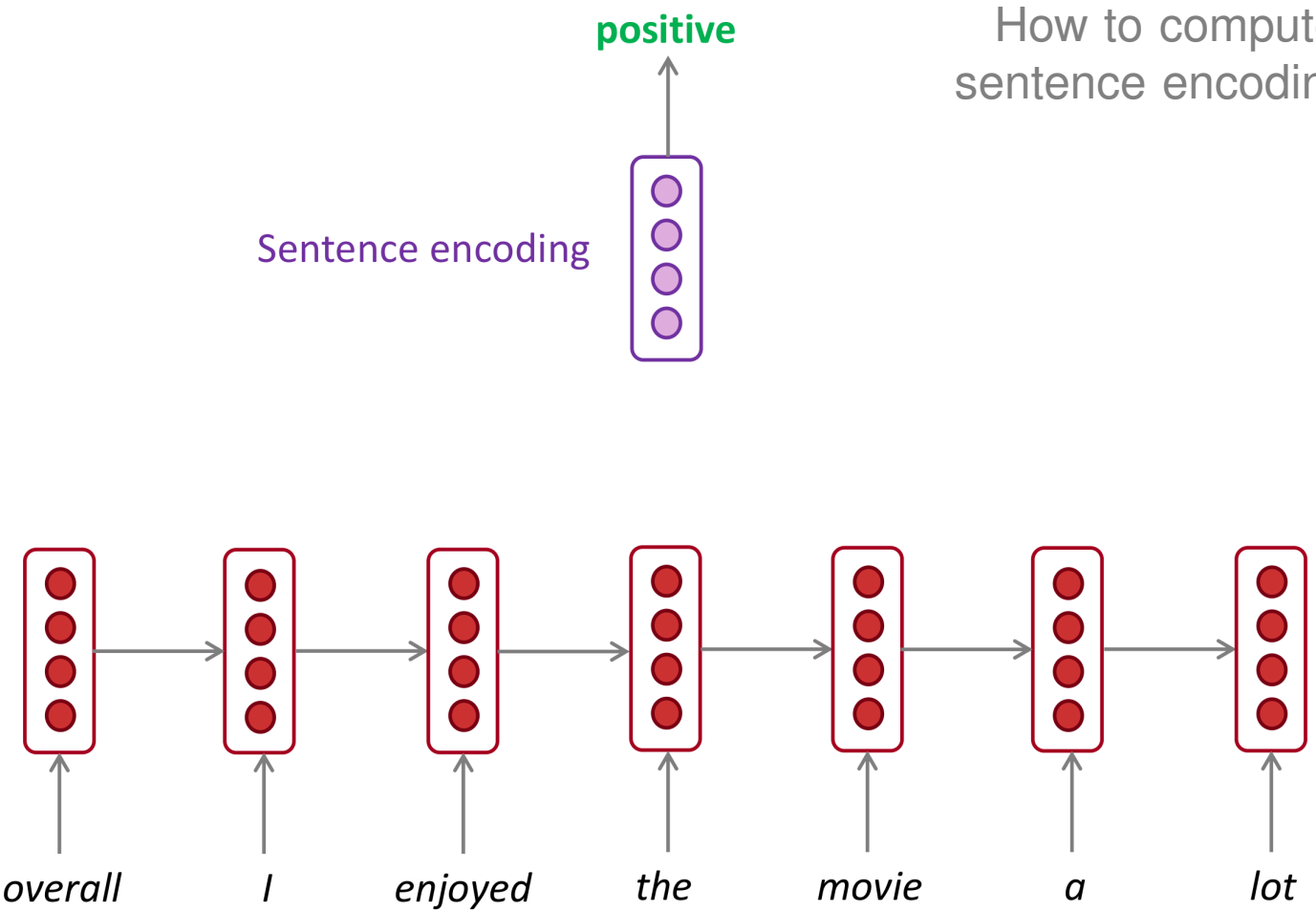
Place each pasta over layers of lumps. Shape mixture into the moderate oven and simmer
until firm. Serve hot in bodied fresh, mustard, orange and cheese.

Combine the cheese and salt together the dough in a large skillet; add the ingredients
and stir in the chocolate and pepper.
```

Source: gist.github.com/nylki/1efbaa36635956d35bcc

RNNs can be used for sentence classification

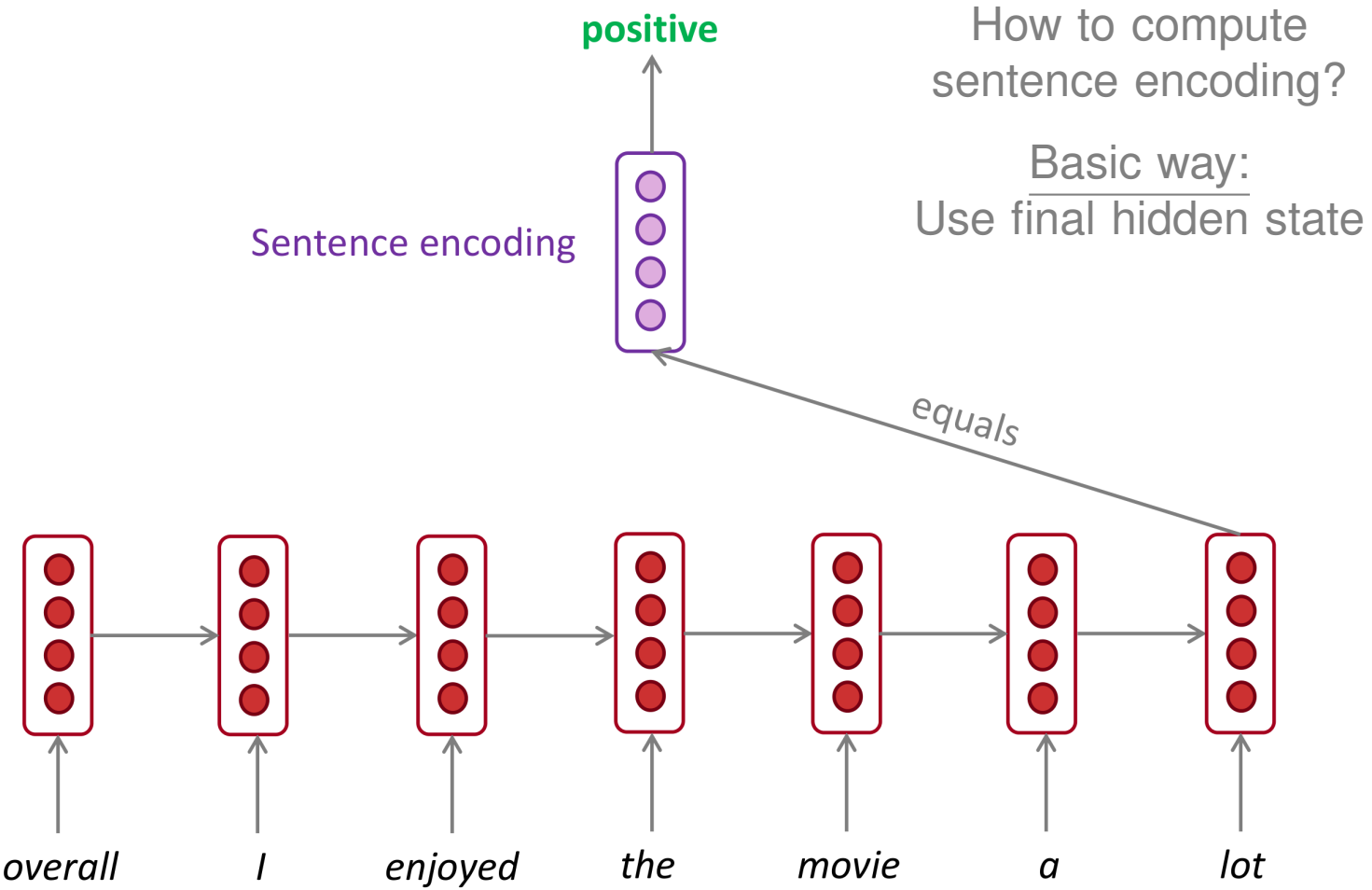
e.g. sentiment classification



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

RNNs can be used for sentence classification

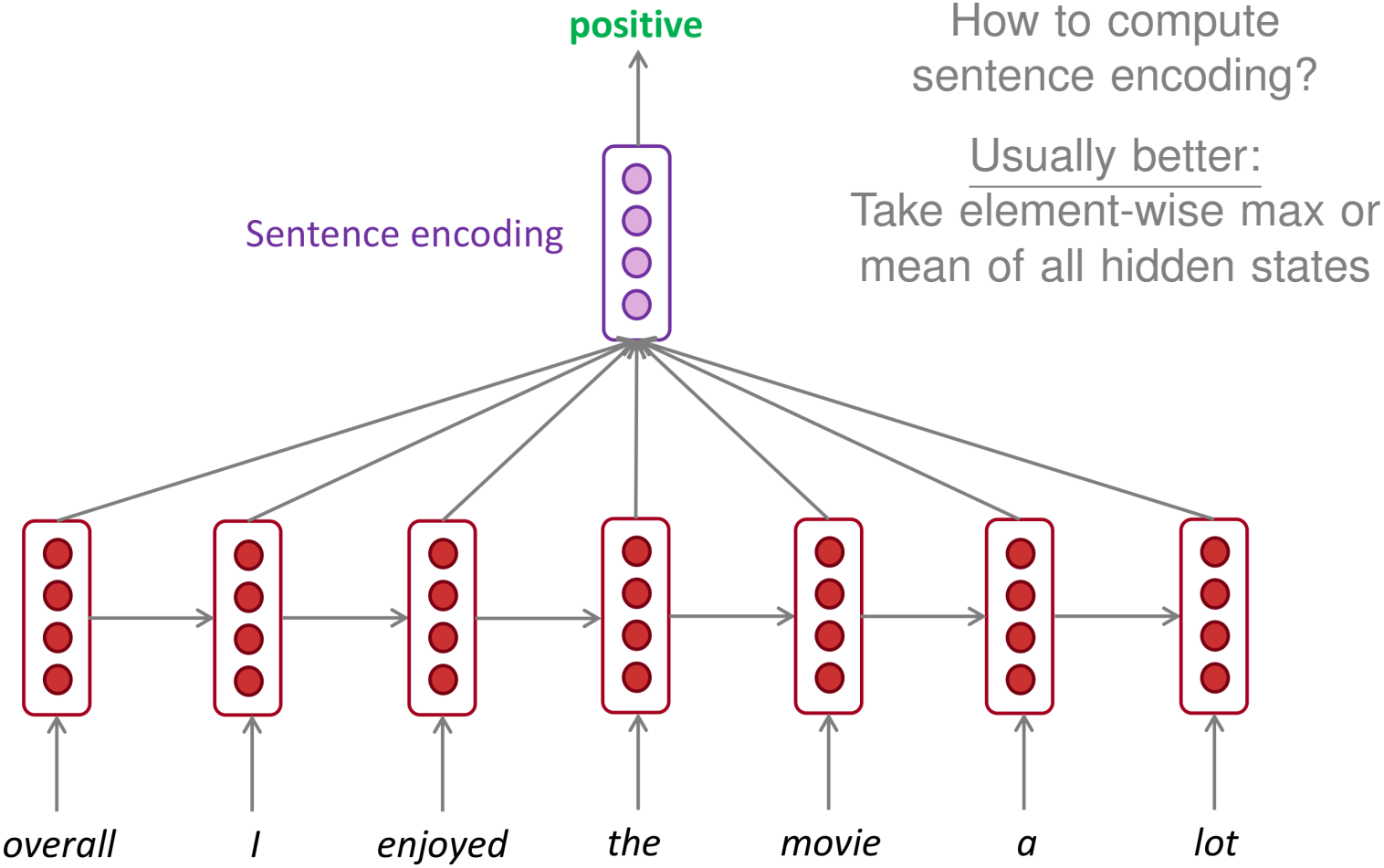
e.g. sentiment classification



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

RNNs can be used for sentence classification

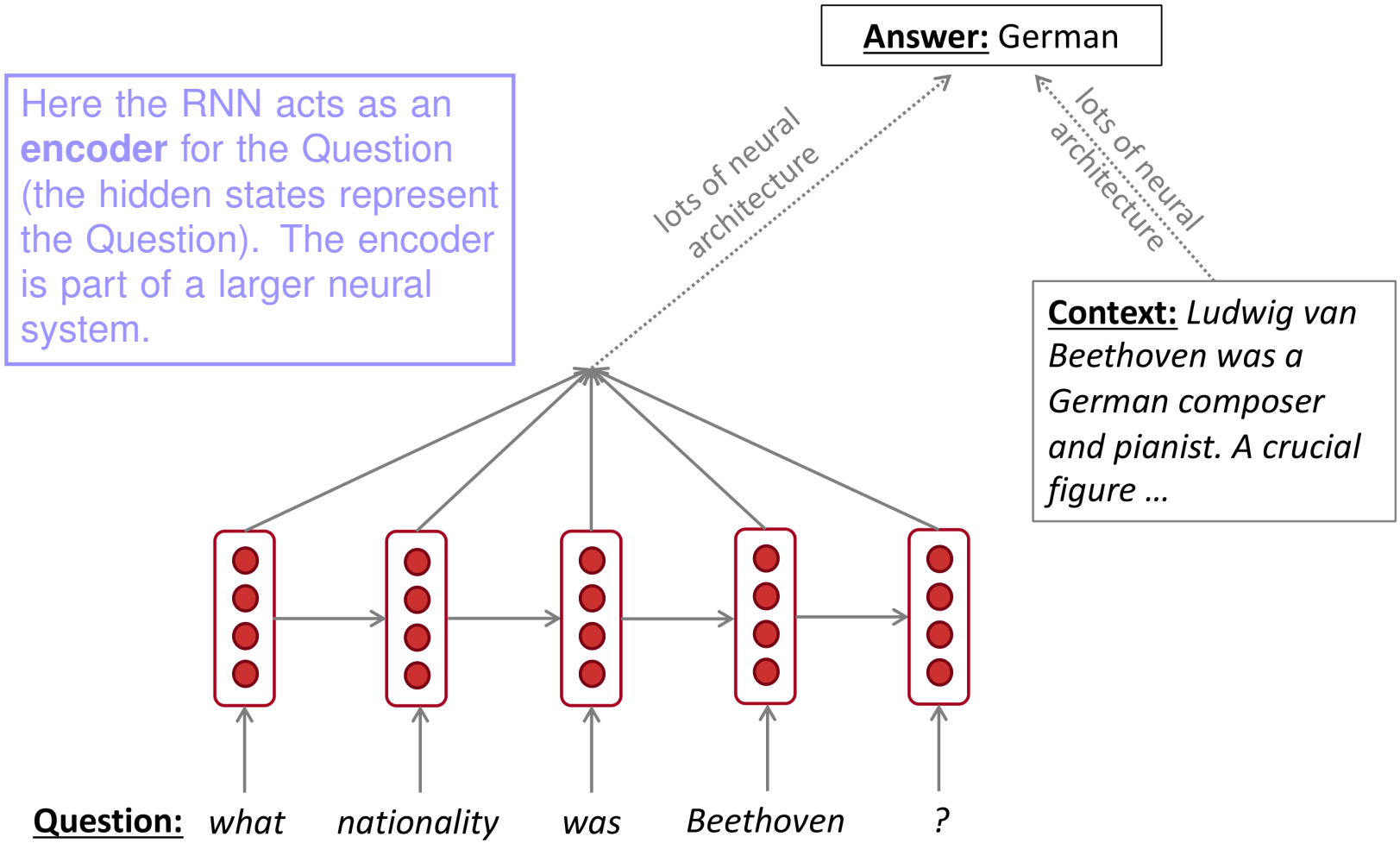
e.g. sentiment classification



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

RNNs can be used as an encoder module

e.g. question answering, machine translation, *many other tasks!*



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

A note on terminology

- The RNN described in this lecture = simple/vanilla/Elman RNN
- Next lecture: You will learn about other RNN flavors like **GRU** and **LSTM** and multi-layer RNNs
- By the end of the course: You will understand phrases like "stacked bidirectional LSTM with residual connections and self-attention"

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Evaluating Language Models

- The standard evaluation metric for Language Models is perplexity.

$$\text{perplexity} = \underbrace{\prod_{t=1}^{T} \left(\frac{1}{P_{\text{LM}}\left(\boldsymbol{x}^{(t+1)} \mid \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}\right)} \right)}_{\text{Inverse probability of corpus, according to Language Model}}^{1/T} \leftarrow \text{Normalized by number of words}$$

- This is equal to the exponential of the cross-entropy loss $J(\theta)$:

$$= \prod_{t=1}^{T} \left(\frac{1}{\hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}_{t+1}}} \right)^{1/T} = \exp\left(\frac{1}{T} \sum_{t=1}^{T} -\log \hat{\boldsymbol{y}}^{(t)}_{\boldsymbol{x}_{t+1}} \right) = \exp\left(J(\theta)\right)$$

Lower perplexity is better!

RNNs have greatly improved perplexity

n-gram model →

Increasingly complex RNNs ↓

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

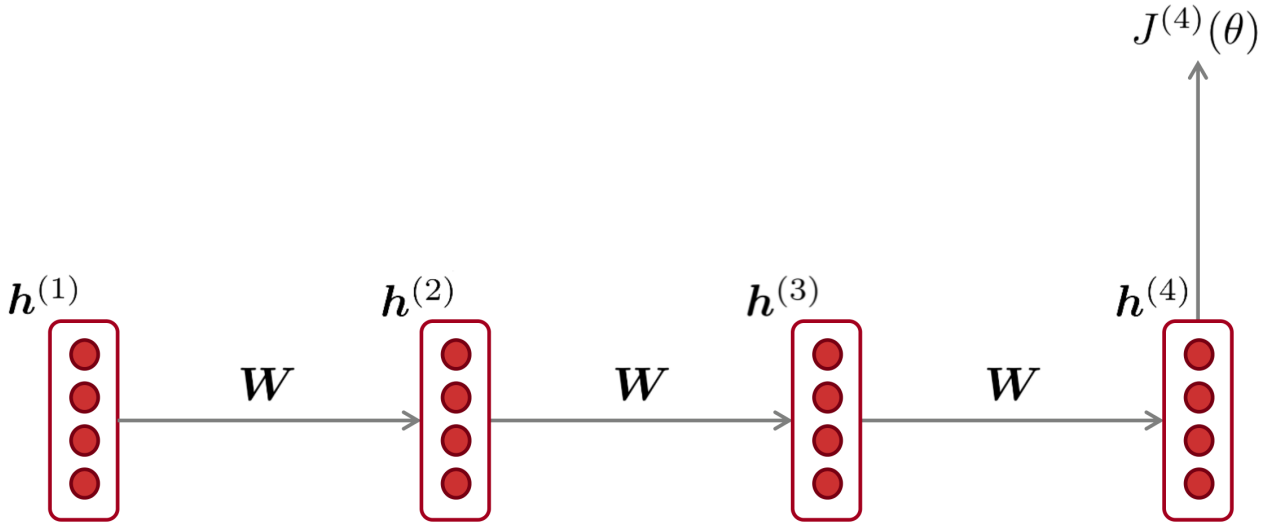
Perplexity improves ↓ (lower is better)

Source: research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words

Content

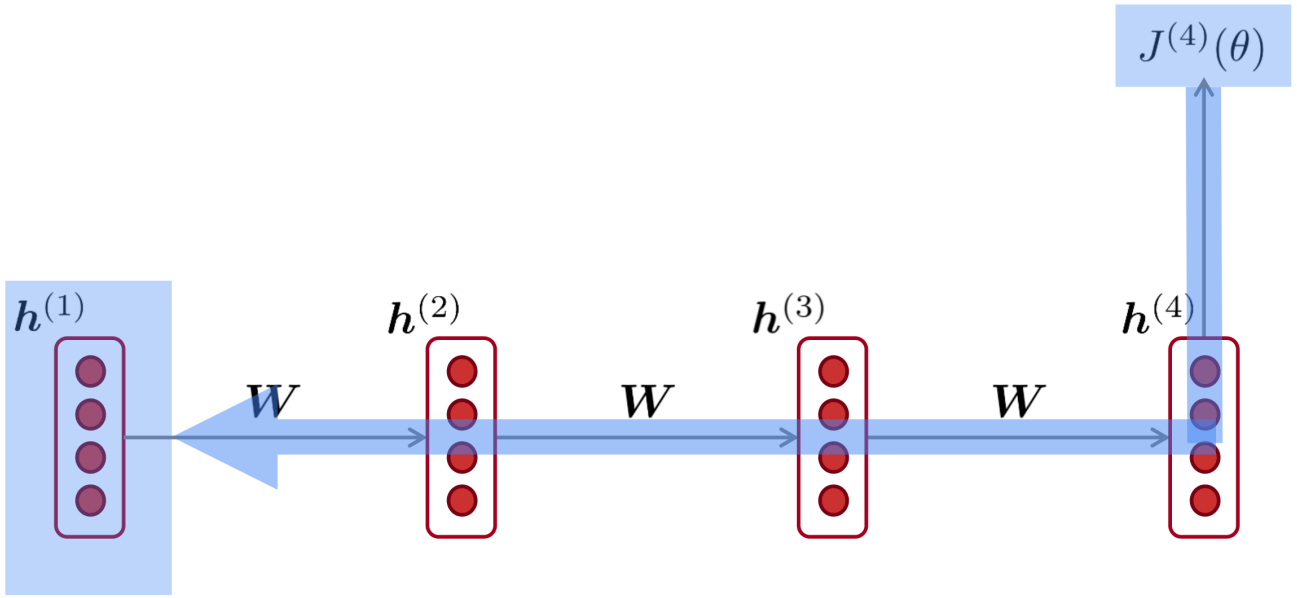
3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Vanishing gradient intuition



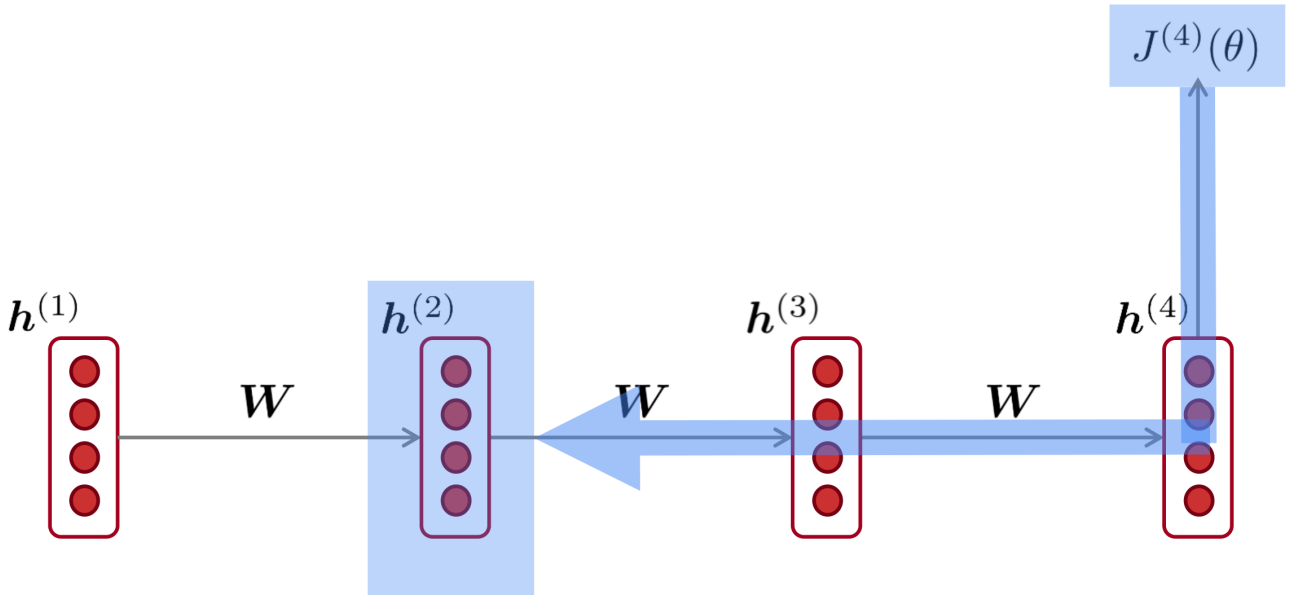
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Vanishing gradient intuition



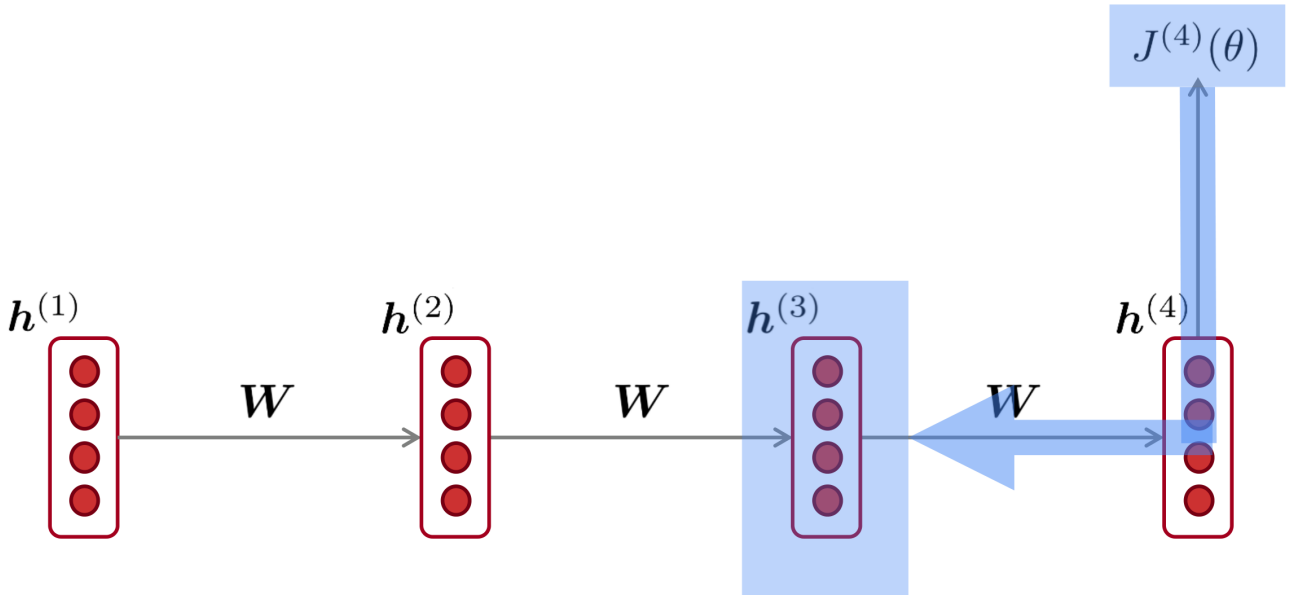
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Vanishing gradient intuition



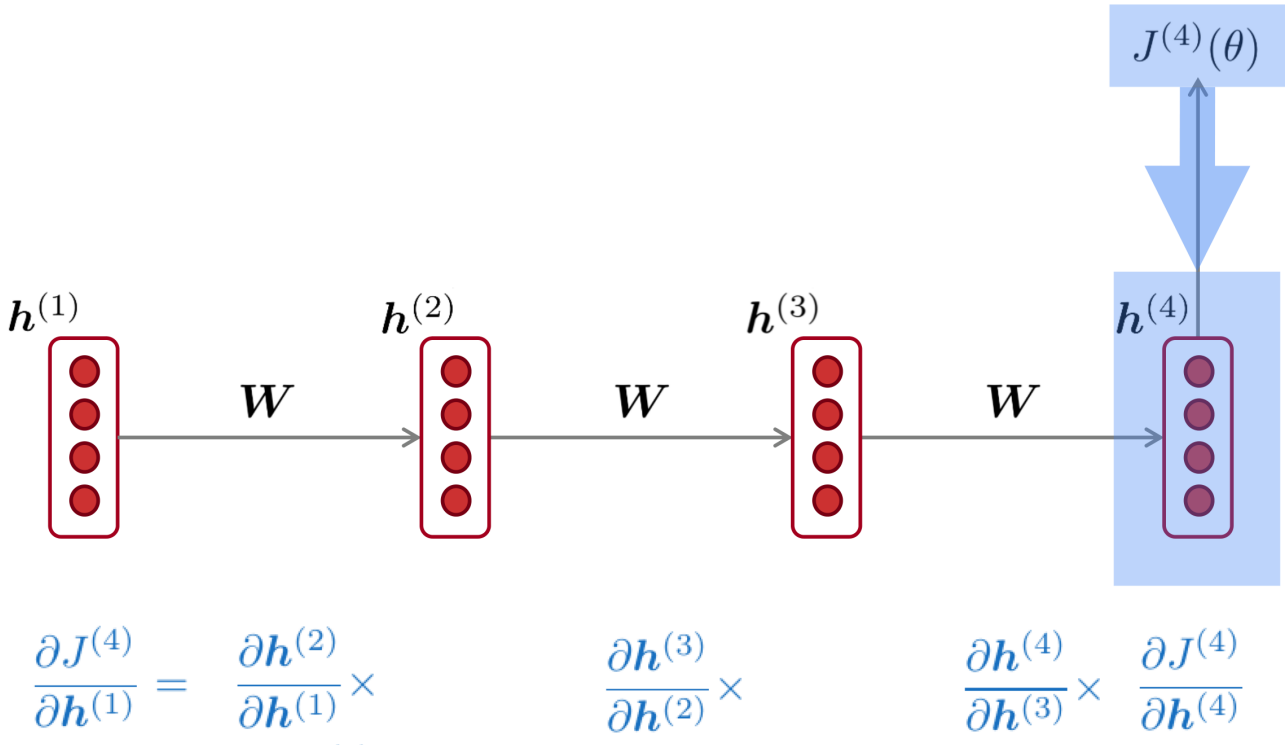
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Vanishing gradient intuition



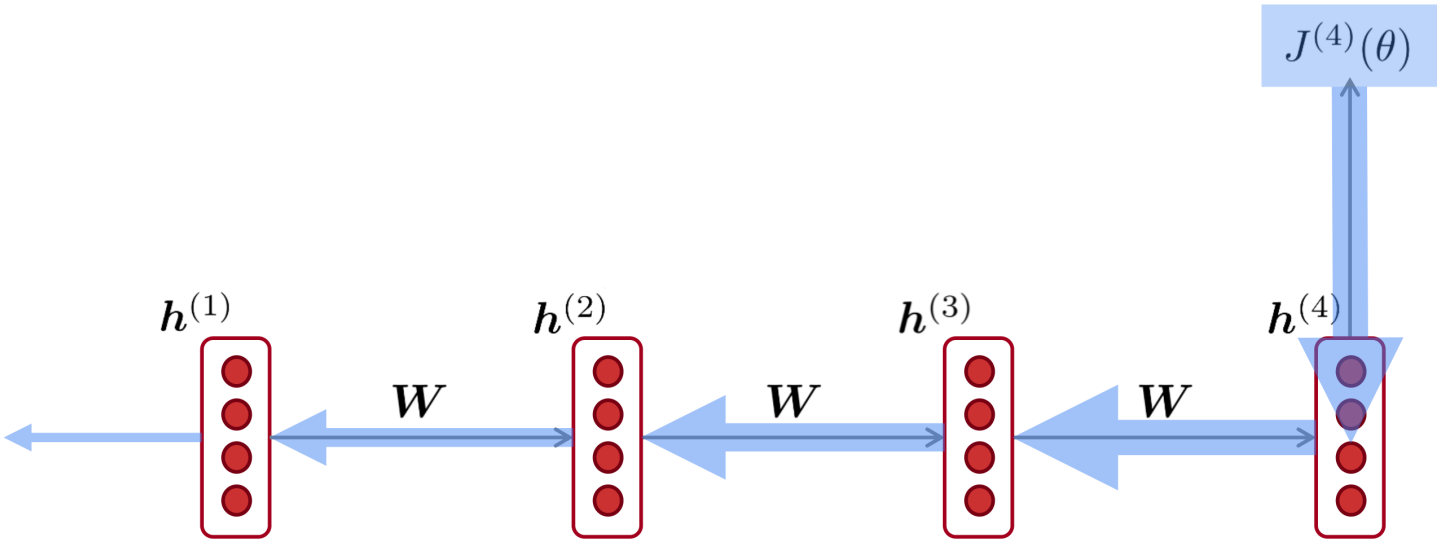
A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Vanishing gradient intuition



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Vanishing gradient intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Vanishing gradient proof sketch

- Recall: $$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_1\right)$$
- Therefore: $$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}} = \operatorname{diag}\left(\sigma'\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_1\right)\right) \boldsymbol{W}_h$$ (chain rule)
- Consider the gradient of the loss $J^{(i)}(\theta)$ on step i, with respect to the hidden state $\boldsymbol{h}^{(j)}$ on some previous step j.

$$\frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(j)}} = \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \prod_{j<t\leq i} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}}$$ (chain rule)

$$= \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \boxed{\boldsymbol{W}_h^{(i-j)}} \prod_{j<t\leq i} \operatorname{diag}\left(\sigma'\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_1\right)\right)$$ (value of $\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(t-1)}}$)

If W_h is small, then this term gets vanishingly small as i and j get further apart

Source: "On the difficulty of training recurrent neural networks", Pascanu et al, 2013. proceedings.mlr.press/v28/pascanu13.pdf

Vanishing gradient proof sketch

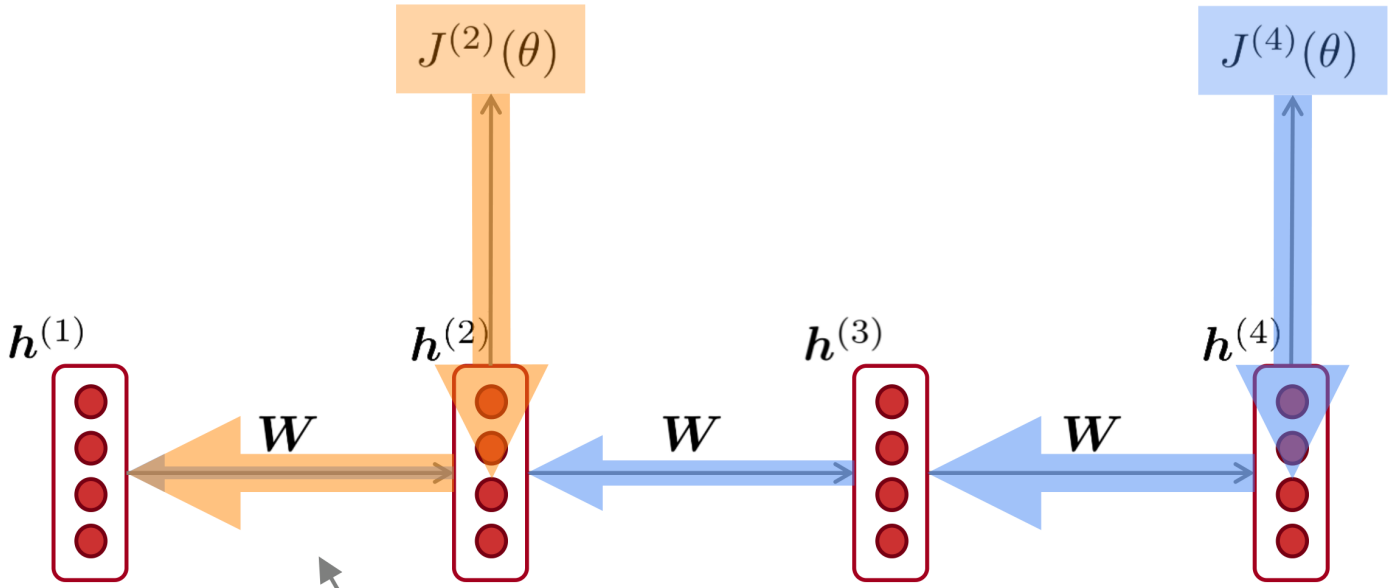
- Consider matrix L2 norms:

$$\left\| \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(j)}} \right\| \leq \left\| \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(i)}} \right\| \|\boldsymbol{W}_h\|^{(i-j)} \prod_{j<t\leq i} \left\| \operatorname{diag}\left(\sigma'\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}_1\right)\right) \right\|$$

- Pascanu et al showed that if the largest eigenvalue of W_h is less than 1, then the gradient $\left\| \frac{\partial J^{(i)}(\theta)}{\partial \boldsymbol{h}^{(j)}} \right\|$ will shrink exponentially
  - Here the bound is 1 because we have sigmoid nonlinearity

Source: "On the difficulty of training recurrent neural networks", Pascanu et al, 2013. proceedings.mlr.press/v28/pascanu13.pdf

Why is vanishing gradient a problem?



Gradient signal from faraway is lost because it's much smaller than gradient signal from close-by.

So model weights are only updated with respect to near effects, not long-term effects.

A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Why is vanishing gradient a problem?

- Another explanation: Gradient can be viewed as a measure of the effect of the past on the future
- If the gradient becomes vanishingly small over longer distances (step t to step $t + n$), then we can't tell whether:
  1. There's no dependency between step t and $t + n$ in the data
  2. We have wrong parameters to capture the true dependency between t and $t + n$

Effect of vanishing gradient on RNN-LM

- **LM task:** *The writer of the books ____* → *is* / *are*
- **Correct answer**: *The writer of the books is planning a sequel*
- **Syntactic recency:** *The writer of the books is* (correct)
- **Sequential recency:** *The writer of the books are* (incorrect)
- Due to vanishing gradient, RNN-LMs are better at learning from sequential recency than syntactic recency, so they make this type of error more often than we'd like [Linzen et al 2016]

"Assessing the Ability of LSTMs to Learn Syntax-Sensitive Dependencies", Linzen et al, 2016. arxiv.org/pdf/1611.01368.pdf

How to fix vanishing gradient problem?

- The main problem is that it's too difficult for the RNN to learn to preserve information over many timesteps.
- In a vanilla RNN, the hidden state is constantly being rewritten:

$$h^{(t)} = f\left(h^{(t-1)}, x^{(t)}\right)$$

- How about a RNN with separate memory?

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
- On step t, there is a hidden state $h^{(t)}$ and a cell state $c^{(t)}$
  - Both are vectors length n
  - The cell stores long-term information
  - The LSTM can erase, write and read information from the cell
- The selection of which information is erased/written/read is controlled by three corresponding gates
  - The gates are also vectors length n
  - On each timestep, each element of the gates can be open (1), closed (0), or somewhere in-between
  - The gates are dynamic: their value is computed based on the current context

"Long short-term memory", Hochreiter and Schmidhuber, 1997,

`https://www.bioinf.jku.at/publications/older/2604.pdf`

Long Short-Term Memory (LSTM)

We have a sequence of inputs $\boldsymbol{x}^{(t)}$, and we will compute a sequence of hidden states $\boldsymbol{h}^{(t)}$ and cell states $\boldsymbol{c}^{(t)}$. On timestep t:

Forget gate: controls what is kept vs forgotten, from previous cell state

$$\boldsymbol{f}^{(t)} = \sigma\left(\boldsymbol{W}_f \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_f \boldsymbol{x}^{(t)} + \boldsymbol{b}_f\right)$$

Input gate: controls what parts of the new cell content are written to cell

$$\boldsymbol{i}^{(t)} = \sigma\left(\boldsymbol{W}_i \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_i \boldsymbol{x}^{(t)} + \boldsymbol{b}_i\right)$$

Output gate: controls what parts of cell are output to hidden state

$$\boldsymbol{o}^{(t)} = \sigma\left(\boldsymbol{W}_o \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_o \boldsymbol{x}^{(t)} + \boldsymbol{b}_o\right)$$

Sigmoid function: all gate values are between 0 and 1

New cell content: this is the new content to be written to the cell

$$\tilde{\boldsymbol{c}}^{(t)} = \tanh\left(\boldsymbol{W}_c \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_c \boldsymbol{x}^{(t)} + \boldsymbol{b}_c\right)$$

Cell state: erase ("forget") some content from last cell state, and write ("input") some new cell content

$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \circ \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \circ \tilde{\boldsymbol{c}}^{(t)}$$

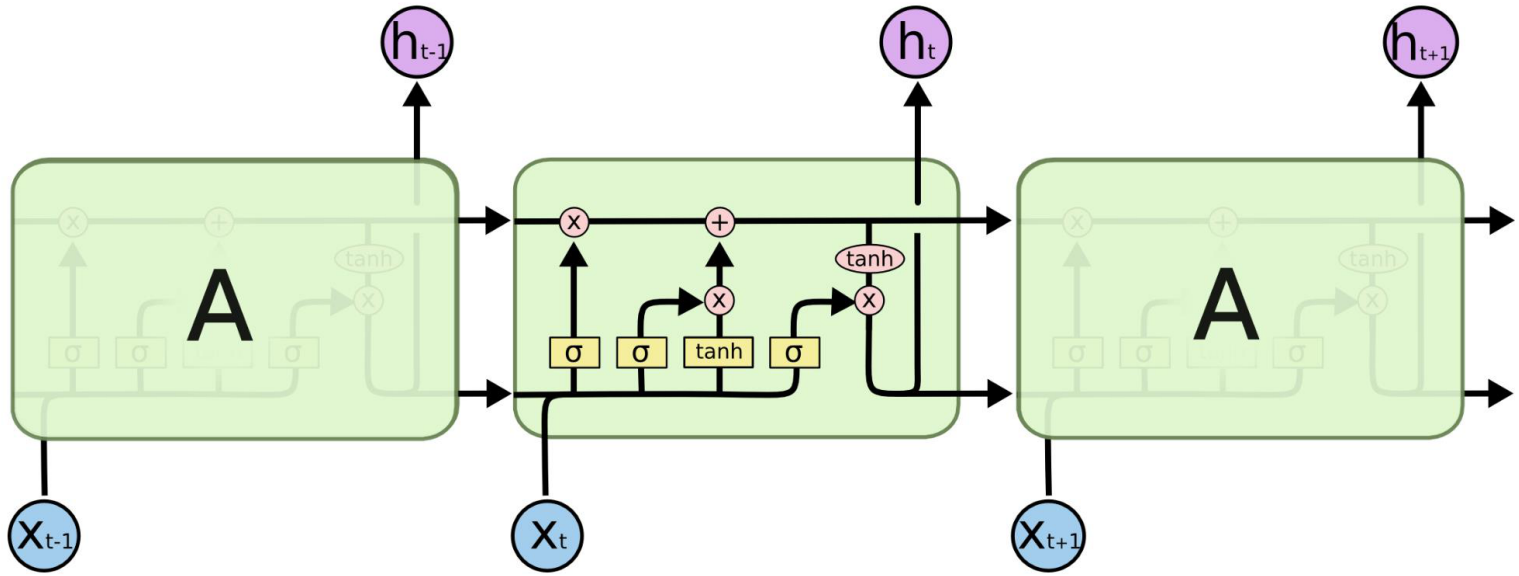
Hidden state: read ("output") some content from the cell

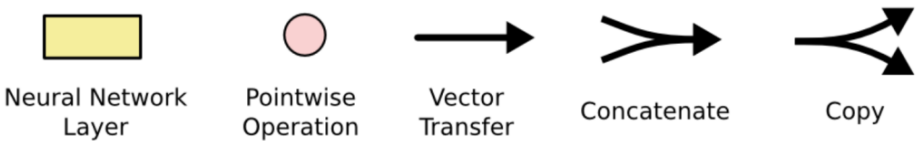
$$\boldsymbol{h}^{(t)} = \boldsymbol{o}^{(t)} \circ \tanh \boldsymbol{c}^{(t)}$$

Gates are applied using element-wise product $\circ$. All these are vectors of same length n

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:

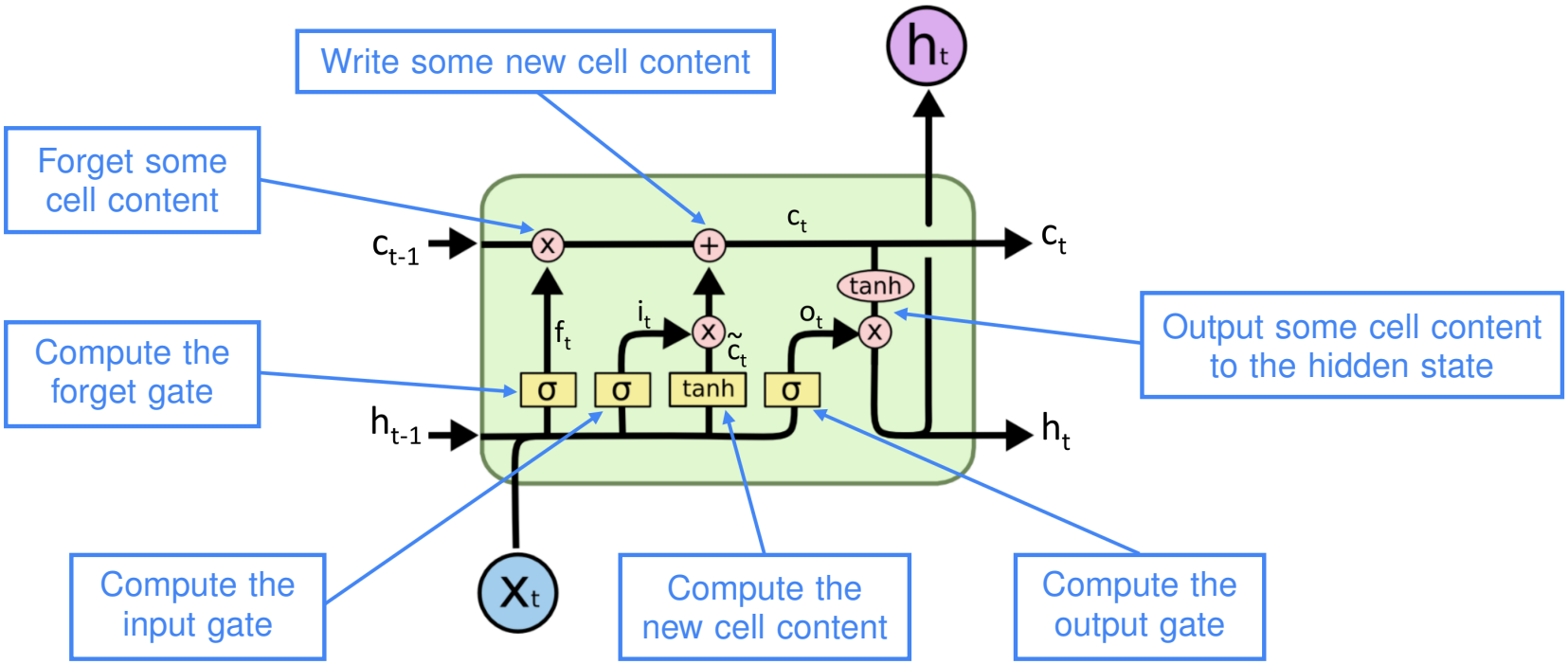




Source: colah.github.io/posts/2015-08-Understanding-LSTMs

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



Source: colah.github.io/posts/2015-08-Understanding-LSTMs

How does LSTM solve vanishing gradients?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps
  - e.g. if the forget gate is set to remember everything on every timestep, then the info in the cell is preserved indefinitely
- By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state
- LSTM doesn't guarantee that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

LSTMs: real-world success

- In 2013-2015, LSTMs started achieving state-of-the-art results
- Successful tasks include: handwriting recognition, speech recognition, machine translation, parsing, image captioning
- LSTM became the dominant approach
- Now (2019), other approaches (e.g. Transformers) have become more dominant for certain tasks
  - For example in WMT (a MT conference + competition):
    - In WMT 2016, the summary report contains “RNN” 44 times
    - In WMT 2018, the report contains “RNN” 9 times and “Transformer” 63 times

Sources: “Findings of the 2016 Conference on Machine Translation (WMT16)”, Bojar et al. 2016; “Findings of the 2018 Conference on Machine Translation (WMT18)”, Bojar et al. 2018

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Gated Recurrent Units (GRU)

- Proposed by Cho et al. in 2014 as a simpler alternative to the LSTM.
- On each timestep t we have input $\boldsymbol{x}^{(t)}$ and hidden state $\boldsymbol{h}^{(t)}$ (no cell state).

Update gate: controls what parts of hidden state are updated vs preserved

$$\boldsymbol{u}^{(t)} = \sigma\left(\boldsymbol{W}_u \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_u \boldsymbol{x}^{(t)} + \boldsymbol{b}_u\right)$$

Reset gate: controls what parts of previous hidden state are used to compute new content

$$\boldsymbol{r}^{(t)} = \sigma\left(\boldsymbol{W}_r \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_r \boldsymbol{x}^{(t)} + \boldsymbol{b}_r\right)$$

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

$$\tilde{\boldsymbol{h}}^{(t)} = \tanh\left(\boldsymbol{W}_h(\boldsymbol{r}^{(t)} \circ \boldsymbol{h}^{(t-1)}) + \boldsymbol{U}_h \boldsymbol{x}^{(t)} + \boldsymbol{b}_h\right)$$

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

$$\boldsymbol{h}^{(t)} = (1 - \boldsymbol{u}^{(t)}) \circ \boldsymbol{h}^{(t-1)} + \boldsymbol{u}^{(t)} \circ \tilde{\boldsymbol{h}}^{(t)}$$

How does this solve vanishing gradient? Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

"Learning Phrase Representations using RNN Encoder–Decoder for SMT", Cho et al. 2014. arxiv.org/pdf/1406.1078v3.pdf

LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used
- The biggest difference is that GRU is quicker to compute and has fewer parameters
- There is no conclusive evidence that one consistently performs better than the other
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data)
- Rule of thumb: start with LSTM, but switch to GRU if you want something more efficient

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Why is exploding gradient a problem?

- The mirror image of vanishing: if the largest eigenvalue of W_h is greater than 1, the gradient can grow exponentially [Pascanu et al, 2013]
- If the gradient becomes too big, then the SGD update step becomes too big:

$$\theta_{\text{new}} = \theta_{\text{old}} - \underbrace{\alpha}_{\text{learning rate}} \underbrace{\nabla_\theta J(\theta)}_{\text{gradient}}$$

- This can cause bad updates: we take too large a step and reach a bad parameter configuration (with large loss)
- In the worst case, this will result in Inf or NaN in your network (then you have to restart training from an earlier checkpoint)

Gradient clipping: solution for exploding gradient

- Gradient clipping: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm: norm clipping

$$\hat{\boldsymbol{g}} \leftarrow \frac{\partial L}{\partial \theta}$$

if $\|\hat{\boldsymbol{g}}\| \geq \mathit{threshold}$ **then**

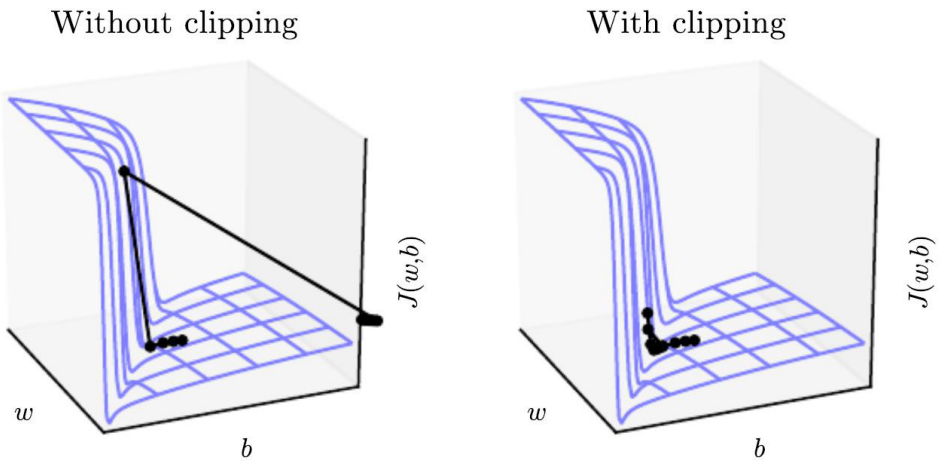
$$\hat{\boldsymbol{g}} \leftarrow \frac{\mathit{threshold}}{\|\hat{\boldsymbol{g}}\|} \hat{\boldsymbol{g}}$$

end if

- Intuition: take a step in the same direction, but a smaller step

Source: "On the difficulty of training recurrent neural networks", Pascanu et al, 2013. proceedings.mlr.press/v28/pascanu13.pdf

Gradient clipping: solution for exploding gradient



- This shows the loss surface of a simple RNN (hidden state is a scalar not a vector)
- The "cliff" is dangerous because it has steep gradient
- On the left, gradient descent takes two very big steps due to steep gradient, resulting in climbing the cliff then shooting off to the right (both bad updates)
- On the right, gradient clipping reduces the size of those steps, so effect is less drastic

Source: "Deep Learning", Goodfellow, Bengio and Courville, 2016. Chapter 10.11.1. deeplearningbook.org/contents/rnn.html

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Is vanishing/exploding gradient just a RNN problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional), especially deep ones.
  - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
  - Thus lower layers are learnt very slowly (hard to train)
  - Solution: lots of new deep feedforward/convolutional architectures that add more direct connections (thus allowing the gradient to flow)

For example:

- Residual connections aka "ResNet"
- Also known as skip-connections
- The identity connection preserves information by default
- This makes deep networks much easier to train

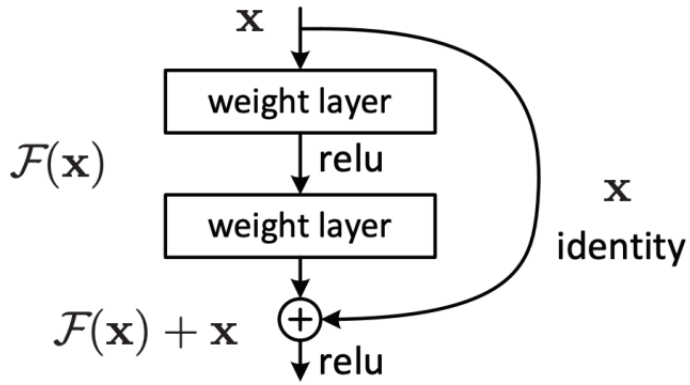


Figure 2. Residual learning: a building block.

"Deep Residual Learning for Image Recognition", He et al, 2015. arxiv.org/pdf/1512.03385.pdf

Is vanishing/exploding gradient just a RNN problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional), especially deep ones.
  - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
  - Thus lower layers are learnt very slowly (hard to train)
  - Solution: lots of new deep feedforward/convolutional architectures that add more direct connections (thus allowing the gradient to flow)

For example:

- Dense connections aka “DenseNet”
- Directly connect everything to everything!

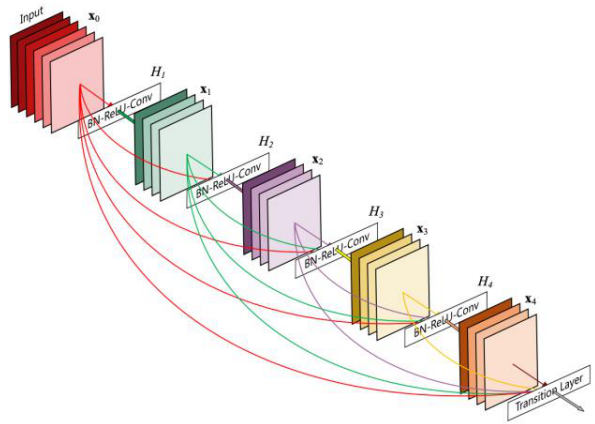


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

“Densely Connected Convolutional Networks”, Huang et al, 2017. arxiv.org/pdf/1608.06993.pdf

Is vanishing/exploding gradient just a RNN problem?

For example: Highway connections aka “HighwayNet”

- Similar to residual connections, but the identity connection vs the transformation layer is controlled by a dynamic gate
- Inspired by LSTMs, but applied to deep feedforward/convolutional networks

- Conclusion: Though vanishing/exploding gradients are a general problem, RNNs are particularly unstable due to the repeated multiplication by the same weight matrix [Bengio et al, 1994]

“Learning Long-Term Dependencies with Gradient Descent is Difficult”, Bengio et al. 1994,

http://ai.dinfo.unifi.it/paolo//ps/tnn-94-gradient.pdf

Is vanishing/exploding gradient just a RNN problem?

For example: Highway connections aka “HighwayNet”

- Similar to residual connections, but the identity connection vs the transformation layer is controlled by a dynamic gate
- Inspired by LSTMs, but applied to deep feedforward/convolutional networks

- Conclusion: Though vanishing/exploding gradients are a general problem, RNNs are particularly unstable due to the repeated multiplication by the same weight matrix [Bengio et al, 1994]

“Learning Long-Term Dependencies with Gradient Descent is Difficult”, Bengio et al. 1994,

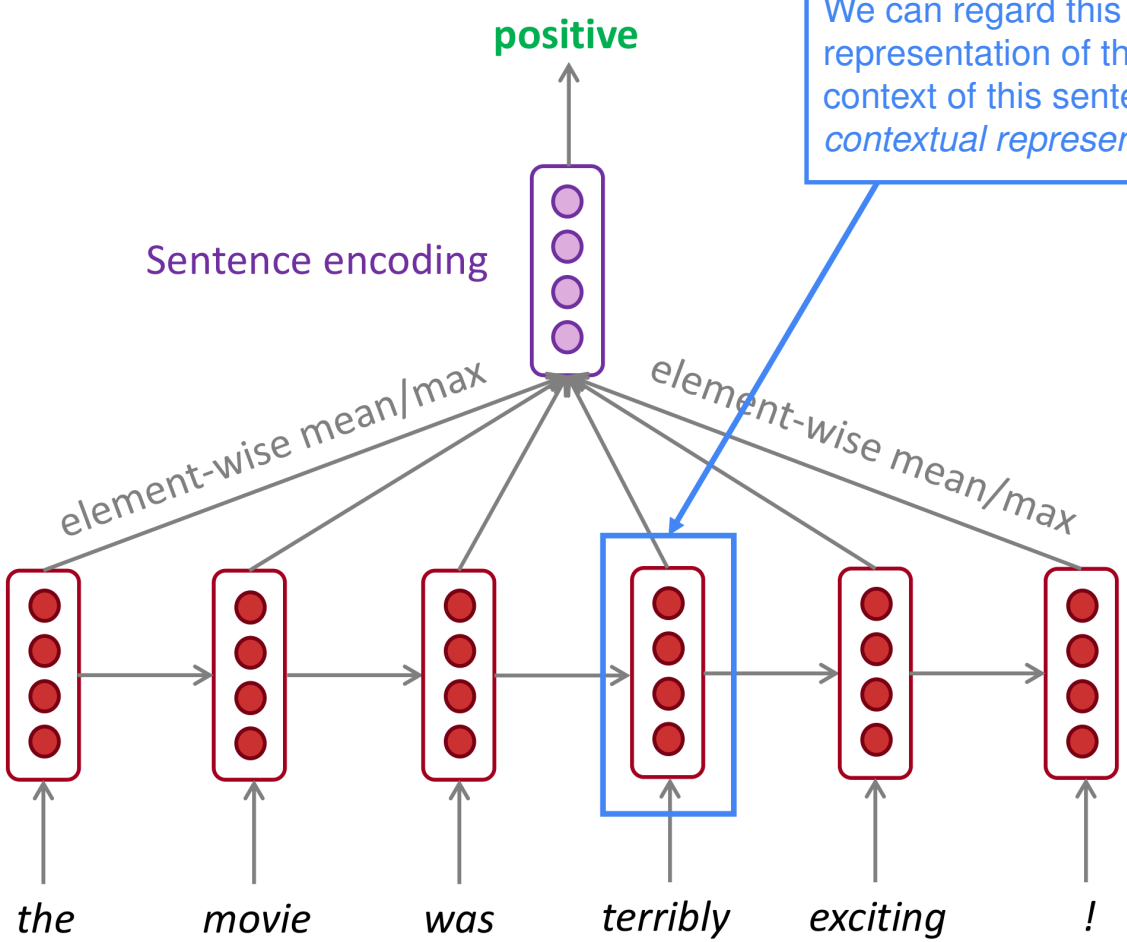
`http://ai.dinfo.unifi.it/paolo//ps/tnn-94-gradient.pdf`

Content

3 Language Models Based on Recurrent Neural Networks
- Recurrent Neural Networks (RNNs)
- Training a neural language model
- Other applications of neural language models
- Evaluation of language models
- Vanishing gradient
- Long Short Term Memory (LSTM)
- Gated Recurrent Units (GRUs)
- Exploding gradient
- Vanishing and exploding gradient for other neural networks
- Bi-directional RNNs and multi-layer RNNs

Bidirectional RNNs: motivation

Task: Sentiment Classification



We can regard this hidden state as a representation of the word "*terribly*" in the context of this sentence. We call this a *contextual representation.*

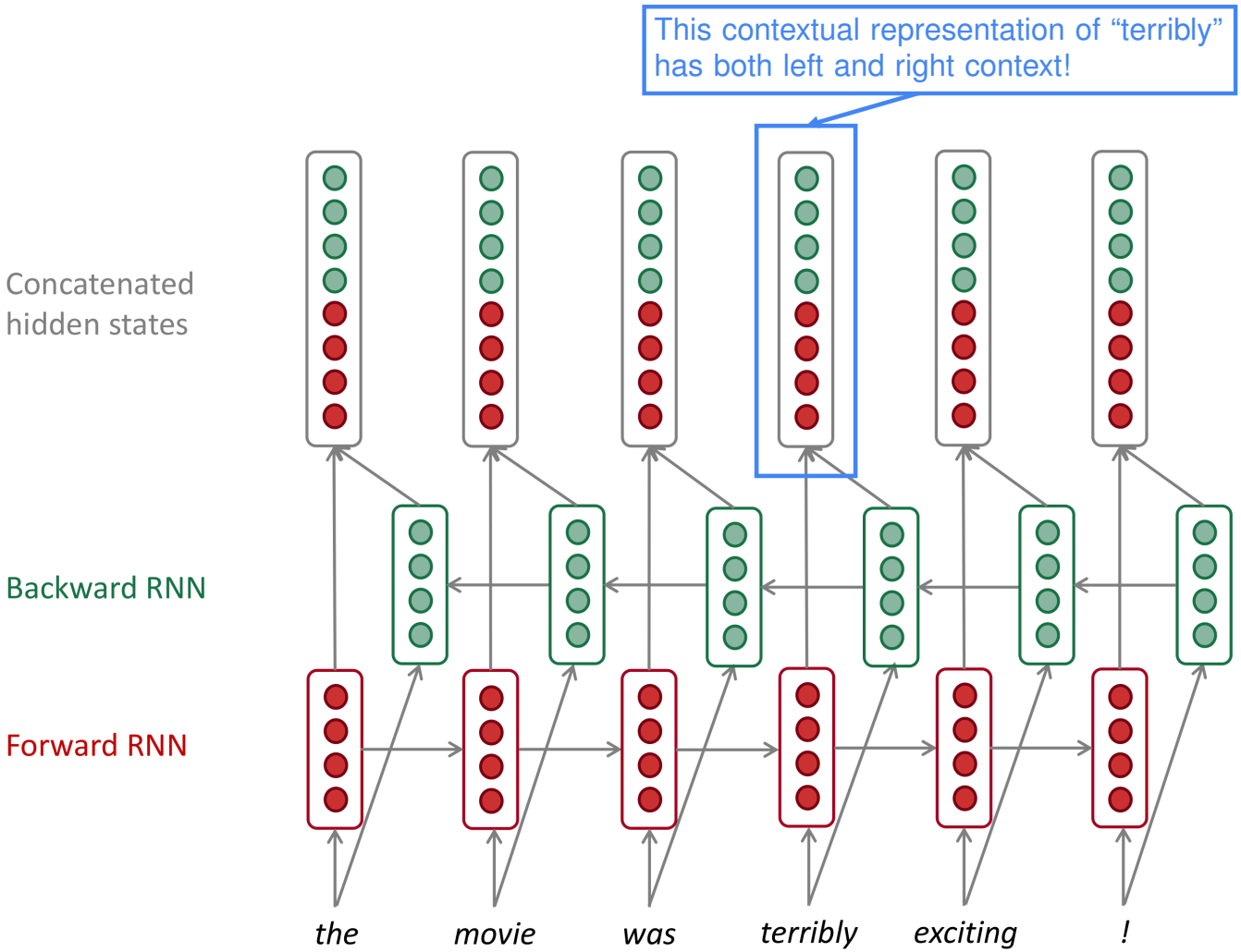
These contextual representations only contain information about the *left* context (e.g. *"the movie was"*).

What about *right* context?

In this example, *"exciting"* is in the right context and this modifies the meaning of *"terribly"* (from negative to positive)

A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Bidirectional RNNs



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Bidirectional RNNs

On timestep t:

This is a general notation to mean "compute one forward step of the RNN" – it could be a vanilla, LSTM or GRU computation.

Forward RNN $\overrightarrow{\boldsymbol{h}}^{(t)} = \mathrm{RNN}_{\mathrm{FW}}\left(\overrightarrow{\boldsymbol{h}}^{(t-1)}, \boldsymbol{x}^{(t)}\right)$

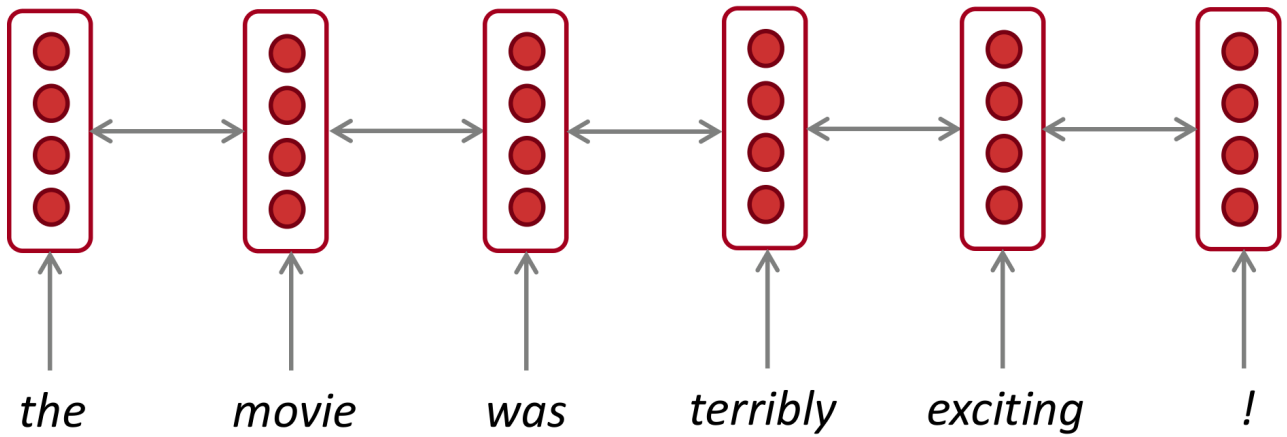
Backward RNN $\overleftarrow{\boldsymbol{h}}^{(t)} = \mathrm{RNN}_{\mathrm{BW}}\left(\overleftarrow{\boldsymbol{h}}^{(t+1)}, \boldsymbol{x}^{(t)}\right)$

Generally, these two RNNs have separate weights

Concatenated hidden states $\boldsymbol{h}^{(t)} = \left[\overrightarrow{\boldsymbol{h}}^{(t)}; \overleftarrow{\boldsymbol{h}}^{(t)}\right]$

We regard this as "the hidden state" of a bidirectional RNN. This is what we pass on to the next parts of the network.

Bidirectional RNNs: simplified diagram



The two-way arrows indicate bidirectionality and the depicted hidden states are assumed to be the concatenated forwards+backwards states.

A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Bidirectional RNNs

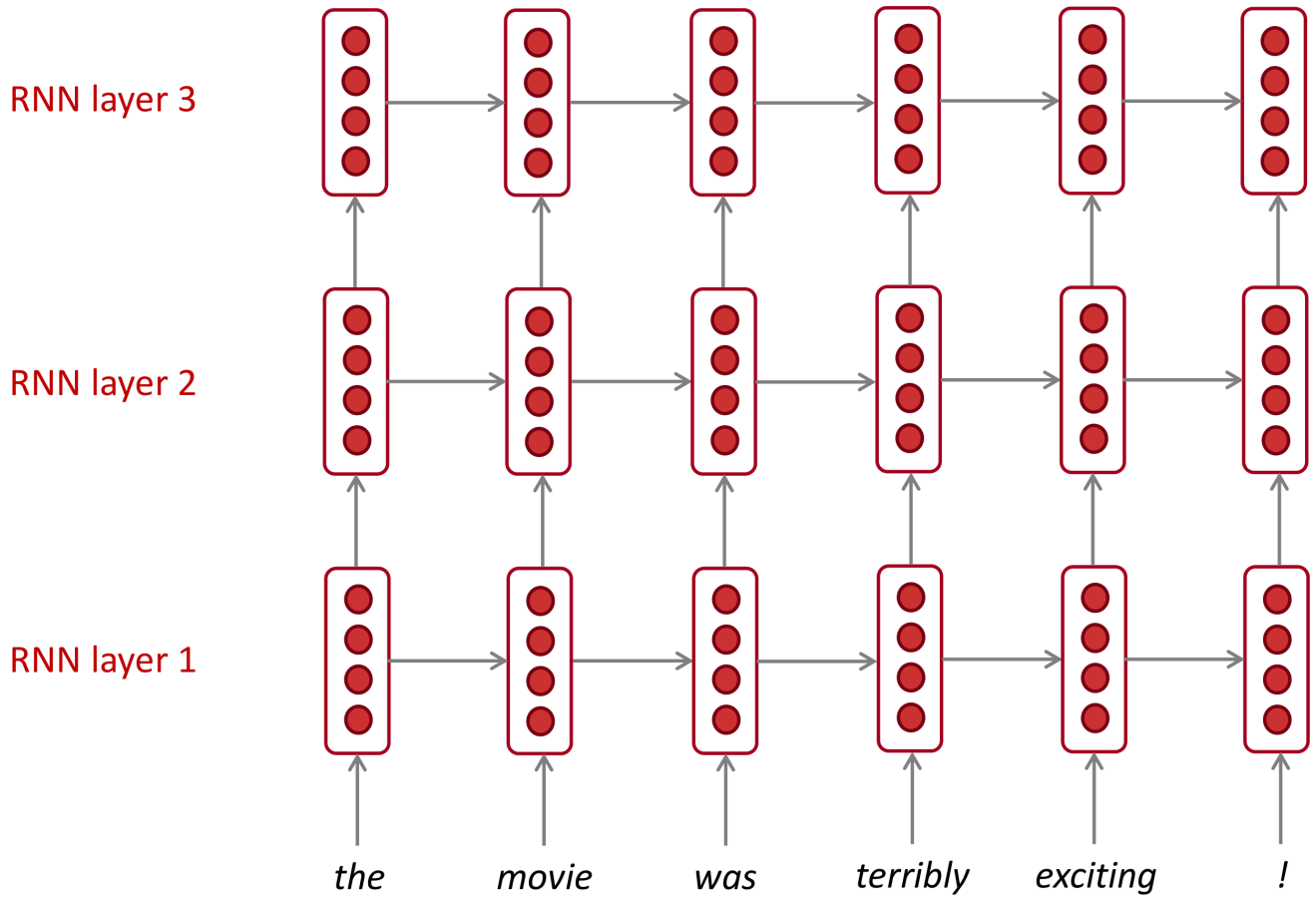
- Note: bidirectional RNNs are only applicable if you have access to the entire input sequence
- They are not applicable to Language Modeling, because in LM you only have left context available
- If you do have entire input sequence (e.g. any kind of encoding), bidirectionality is powerful (you should use it by default)
  - For example, BERT (Bidirectional Encoder Representations from Transformers) is a powerful pretrained contextual representation system built on bidirectionality
  - You will learn more about BERT later in the course!

Multi-layer RNNs

- RNNs are already “deep” on one dimension (they unroll over many timesteps)
- We can also make them “deep” in another dimension by applying multiple RNNs — this is a multi-layer RNN
  - This allows the network to compute more complex representations
- The lower RNNs should compute lower-level features and the higher RNNs should compute higher-level features
- Multi-layer RNNs are also called stacked RNNs

Multi-layer RNNs

The hidden states from RNN layer i are the inputs to RNN layer $i+1$



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

Multi-layer RNNs in practice

- High-performing RNNs are often multi-layer (but aren't as deep as convolutional or feed-forward networks)
  - For example: In a 2017 paper, Britz et al find that for Neural Machine Translation, 2 to 4 layers is best for the encoder RNN, and 4 layers is best for the decoder RNN
- However, skip-connections/dense-connections are needed to train deeper RNNs (e.g. 8 layers)
  - Transformer-based networks (e.g. BERT) can be up to 24 layers
  - You will learn about Transformers later; they have a lot of skipping-like connections

"Massive Exploration of Neural Machine Translation Architectures", Britz et al, 2017,

`https://arxiv.org/pdf/1703.03906.pdf`

Content

1 Language Models (LMs)

2 N-gram Language Models

3 Language Models Based on Recurrent Neural Networks

4 Sequence Tagging

Named Entity Recognition (NER)

- A very important sub-task: **find** and **classify** names in text, for example:

The decision by the independent MP Andrew Wilkie to withdraw his support for the minority Labor government sounded dramatic but it should not further threaten its stability. When, after the 2010 election, Wilkie, Rob Oakeshott, Tony Windsor and the Greens agreed to support Labor, they gave just two guarantees: confidence and supply.

Named Entity Recognition (NER)

- A very important sub-task: **find** and **classify** names in text, for example:

The decision by the independent MP Andrew Wilkie to withdraw his support for the minority Labor government sounded dramatic but it should not further threaten its stability. When, after the 2010 election, Wilkie, Rob Oakeshott, Tony Windsor and the Greens agreed to support Labor, they gave just two guarantees: confidence and supply.

PER person **ORG** organization **DATE** date

Named Entity Recognition (NER)

- The uses:
  - Named entities can be indexed, linked off, etc.
  - Sentiment can be attributed to companies or products
  - A lot of IE relations are associations between named entities
  - For question answering, answers are often named entities.
- Concretely:
  - Many web pages tag various entities, with links to bio or topic pages, etc.
  - Reuters' OpenCalais, Evri, AlchemyAPI, Yahoo's Term Extraction, ...
  - Apple/Google/Microsoft/...smart recognizers for document content

Three approaches to NER

- Hand-written regular expressions
- Using classifiers
- Sequence models

Hand-written Patterns for Information Extraction

- If extracting from automatically generated web pages, simple regex patterns usually work.
  - Amazon page

    ```
    <div class="buying"><h1 class="parseasinTitle">
    <span id="btAsinTitle" style="">(.*?)</span></h1>
    ```
- For certain restricted, common types of entities in unstructured text, simple regex patterns also usually work.
  - Finding (US) phone numbers

    ```
    (?:\(?[0-9]{3}\)?[ -.])?[0-9]{3}[ -.]?[0-9]{4}
    ```

The Named Entity Recognition Task

- Task: Predict entities in a text

ORG	ORG	O	PER	PER	O	ORG	:
Foreign	Ministry	spokesman	Shen	Guofang	told	Reuters	:

PER person **ORG** organization **O** outside (not an entity)

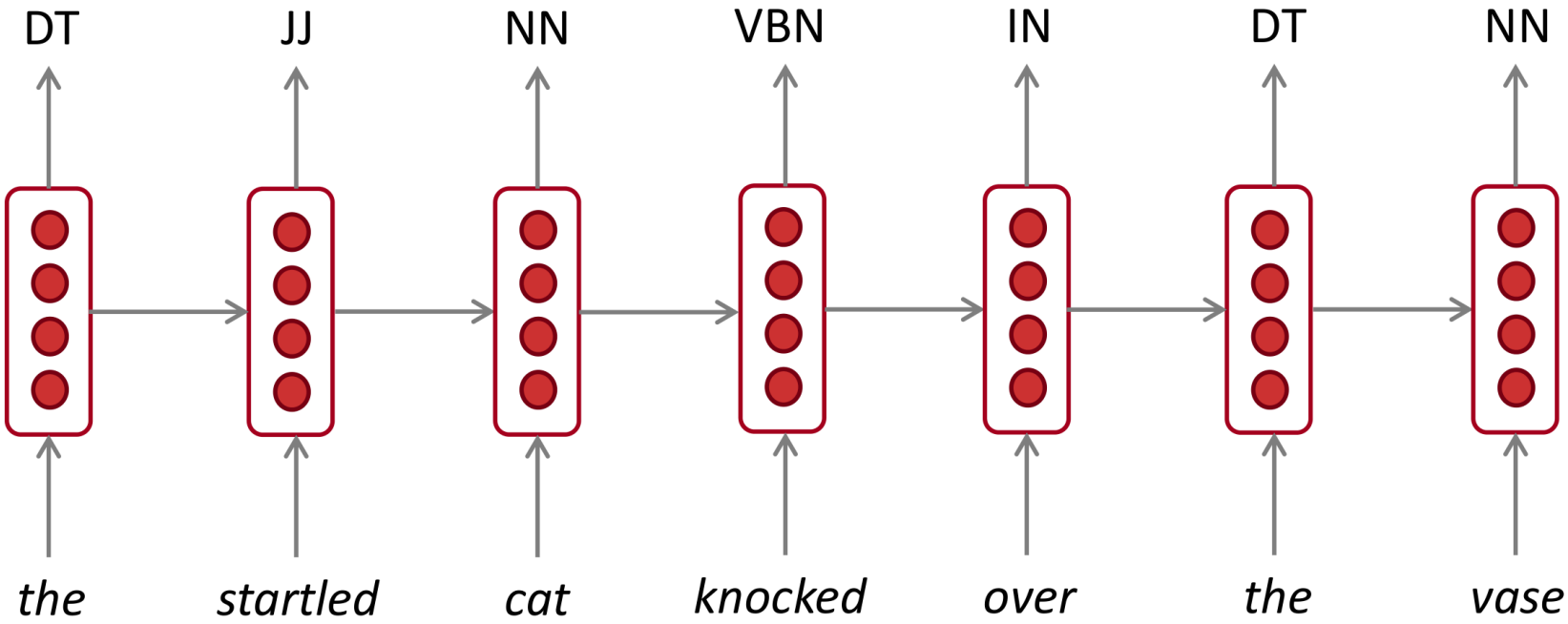
Standard evaluation is per entity, not per token

Encoding classes for sequence labeling

	Fred	showed	Sue	Mengqiu	Huang	's	new	painting
IO	**PER**	O	**PER**	**PER**	**PER**	O	O	O
BIO	**B-PER**	O	**B-PER**	**B-PER**	**I-PER**	O	O	O

RNNs can be used for tagging

e.g. part-of-speech tagging, named entity recognition



A figure from Christopher Manning, Natural Language Processing with Deep Learning, Stanford U. CS224n

The ML sequence model approach to NER

Training

1. Collect a set of representative training documents
2. Label each token for its entity class or other (O)
3. Design feature extractors appropriate to the text and classes
4. Train a sequence classifier to predict the labels from the data

Testing

1. Receive a set of testing documents
2. Run sequence model inference to label each token
3. Appropriately output the recognized entities

Content

1. Language Models (LMs)
2. N-gram Language Models
3. Language Models Based on Recurrent Neural Networks
4. Sequence Tagging